Cloud-Native Agentic AI for Personalised Portfolio Decision Support: System Design and Evaluation Framework

Curtis Mellsop

Table of Contents

1. Introduction 1
2. Thesis Specification 2
   2.1 System Overview 2
   2.2 Technical Overview 2
   2.3 Thesis Aim 2
   2.4 Research Questions 3
   2.5 System Scope, Objectives, and Requirements 3
   2.6 Inputs and Outputs 7
   2.7 Specification Summary 8
3. Literature Review 9
   3.1 Portfolio Decision Support 9
   3.2 Investor Profiling and Portfolio Suitability 11
   3.3 Machine Learning Lifecycle for Financial Prediction 13
   3.4 Agentic AI 16
   3.5 Cloud Deployment Architectures 18
   3.6 Monitoring and Performance Testing 21
   3.7 Research Summary 23
4. Experimental Evaluation 24
   4.1 Experimental Design 24
   4.2 Architecture Experiments 28
   4.3 Overall Findings and Design Implications 35
5. Design Plan 36
   5.1 Overall System Design 36
   5.2 User Profile and Portfolio Management 37
   5.3 Market-Data Pipeline and Predictive Models 38
   5.4 Agentic Decision Process and Portfolio Tools 39
   5.5 Recommendation Output and User Interface 41
   5.6 Data and Model Management, and System Integration 41
   5.7 Deployment, Scaling, and Observability 42
   5.8 Design Overview 43
6. Evaluation Plan 44
   6.1 Evaluation Approach and Methodology 44
   6.2 Functional and Integration Verification 45
   6.3 Predictive Model Evaluation 45
   6.4 Agentic Decision and Recommendation Evaluation 46
   6.5 System Performance Evaluation 47
   6.6 Evaluation Criteria and Requirements Traceability 48
7. Conclusion 50
8. References 51

1. Introduction

Investing has become increasingly accessible to the everyday user through the growth of online platforms and the amount of easily obtainable financial information. However, this ease of access does not necessarily make investment decisions easier for the everyday user. They still need to be able to understand and interpret this information, which is particularly difficult as an ideal investment strategy is not entirely determined by risk and return alone. It should also reflect the user's own personal investment goals. As a result, there is no universal strategy that is tailored towards all investors. This creates a need for tools that can relate personal investment goals to current market information and provide useful and personalised decision support. This idea forms the basis of the proposed system considered throughout this report.

The system will combine user information with market data to make informed decisions, rather than treating each one independently. Predictive models will process market data into asset-level signals, which will then be compared with user profiles through an agentic decision process to generate suitable portfolios. The resulting recommendations will be ranked and supported by clear explanations of why they may be appropriate for the user. The purpose of the system is to provide decision support, but not to make any trades on the user's behalf. This is why it is important for the system to make it clear how each recommendation relates to their personal goals and circumstances. Bringing these services together, however, creates a wider technical challenge around how the system should be structured, deployed, and evaluated.

The system is made up of different types of workloads, rather than one simple application flow. There is the user-request side, which is more latency-sensitive, as there are several stages that need to be completed before recommendations can be returned. Then there is the market-data side, which processes larger background workloads, and as such is more concerned with throughput and scaling to keep the information current. The system also contains a variety of tools and components, each with their own usage requirements. Because of this, one architectural approach is unlikely to be suitable for every component. This creates uncertainty surrounding how best to deploy the system and its individual parts, which is something that is examined further in this project.

This project is not about building the system but instead acts as the planning and justification stage. The purpose is to define what the system should achieve, investigate the research needed to support its design, and test areas where architectural decisions cannot be made confidently from literature alone. The experiments will provide empirical results directly related to the system itself and the potential deployment approaches. These findings can then be combined with the wider research to create a complete system design and evaluation plan for the thesis. As such, this project provides the foundation needed before development of the final system can begin.

The remainder of this report follows this planning process. Section 2 first defines the specification for the intended thesis system, including its scope, objectives, requirements, and expected inputs and outputs. Section 3 then reviews the literature and Section 4 evaluates the main architectural options through a series of controlled experiments. These findings are brought together in Section 5 to define the proposed system design, before Section 6 establishes the evaluation plan that will be used to assess the completed system during the thesis.

2. Thesis Specification

The intention of this project is to provide the justification and planning foundation for an actual investment portfolio recommendation system, which will be built as part of the thesis. This section provides the specification for this system, including the research directives, scope, requirements, and other fundamental aspects. The intent is for this section to say exactly what the system must and should do, but not to make any design decisions. As such, the section will serve as both a reference point for building the design of the system later in this project, and to gauge an idea of what must be researched and experimented in order to build this design. Everything discussed in this section relates specifically to the intended thesis, rather than to the current preparatory project.

2.1 System Overview

The purpose of the thesis is to design and evaluate a cloud-native portfolio decision-support system to help everyday users make more informed investment decisions. Many existing financial platforms provide users with access to investment products, but they often assume users already understand how to interpret financial information or decide which actions are suitable for them. This thesis takes a different approach by focusing on how a system can turn a user's goals, preferences, constraints, and risk tolerance into clear portfolio insights and recommendations.

The proposed system is not intended to invest money or execute trades on behalf of the user. It is designed as a decision-support tool to help users better understand their current position and potential investment options. A user will be able to create an investment profile containing information such as their goals, constraints and preferences. This information can then be processed and compared with market data to produce a personalised portfolio analysis and provide ranked recommendations. The recommendations will also be explainable, so that users can understand why a particular asset or portfolio adjustment may be appropriate for their situation.

2.2 Technical Overview

From a technical perspective, the thesis is centred on building a cloud-native system that connects the main stages of the portfolio recommendation process into a single workflow. User profile information and market data will be transformed into decision-support information that can be used as part of the agentic pipeline to generate personalised portfolio recommendations. This is one of the main areas where the thesis differs from a standard recommendation application: the model output will not be treated as the final decision, but as one input into a wider agentic reasoning process that also considers user preferences to create tailored recommendations.

The thesis will also examine the wider technical environment required to support this type of system in practice. This includes the choice of cloud infrastructure, deployment approach, supporting tools, monitoring strategy, data handling process, and evaluation method. While the current project focuses primarily on comparing potential architectural approaches for the system, the thesis will use this work as a foundation for a broader evaluation and optimization of the systems performance. This will allow the final system to be assessed not only in terms of recommendation suitability but also in terms of how well the chosen design supports aspects such as scalability and observability.

2.3 Thesis Aim

The primary aim of the thesis is to implement and evaluate a cloud-native agentic AI system for personalized portfolio decision support based on the design and evaluation plan laid out in this project.

The thesis will evaluate whether agentic reasoning can support suitable, personalised, and explainable portfolio recommendations. The technical behaviour of the system will also be evaluated to determine whether the chosen cloud-native implementation can support the performance and operational requirements of this type of decision-support system.

2.4 Research Questions

Research for the thesis will look to answer one primary two-part question:

How can a cloud-native agentic AI system be designed to provide accurate personalised portfolio decision support using user information in conjunction with market data? And which technological choices can best improve the performance of such a system?

To answer this question, the thesis considers the following supporting research questions:

1. What information must be collected to create a full user profile that best represents a user's preferences and constraints in relation to their investment strategy?
2. How can market data and predictive models be used to produce asset-level information that supports portfolio decision-making?
3. What information and tools are required for an agentic decision-support process to form relevant portfolios and assess their suitability?
4. How can the system produce explainable portfolio metrics, risk assessments, and suggested actions for everyday users?
5. How can the choice of a cloud-native architecture support the deployment, scaling, monitoring, and overall requirements of an agentic AI portfolio decision-support system?
6. What trade-offs exist between architectural complexity, system performance, reliability, observability, and resource usage when deploying this type of system?

2.5 System Scope, Objectives, and Requirements

This section defines the boundaries of the system. It details in-scope areas to be explored and out-of-scope topics that won't be explored. The objectives define the system's broader goals within this boundary, from which the system's specific requirements can be derived to meet these objectives.

2.5.1 In Scope

Area	Included in Scope
User Profile Management	Users can define their investment preferences. This includes their goals, risk tolerance, budget, and any constraints.
Portfolio Holdings	Users can input and update their current portfolio holdings. The system will keep a record of these holdings for each user.

Market Data Processing	The system will be able to handle large streams of periodical aggregated market data. This data is engineered into features which are used to train/update predictive models.
Agentic Decision Logic	An agentic pipeline consisting of one or multiple autonomous agent/s, and a variety of assisting tools and resources, will be used to create personalised portfolios.
Portfolio Analysis	The system will calculate relevant portfolio metrics for each recommendation, such as allocation, exposure, risk and diversification.
Recommendations	The system will generate ranked portfolio recommendations which contain easy-to-understand insights and metrics. This will be provided through a user-facing interface which provides explanations for each recommendation
Cloud-Native Deployment	The system will be deployable in a cloud-native environment and evaluated against relevant architectural, infrastructure, and operational requirements.
System Evaluation	A full systems performance evaluation will be undergone on the system to ensure it meets the requirements and to ensure that it is operating efficiently.

Table 2.1: In-Scope System Areas

2.5.2 System Objectives

The system objectives define the main goals that the system should achieve within the scope outlined above. The main objectives are as follows:

1. **Develop an end-to-end portfolio decision-support workflow.**

The system should support the full process from user information and market data collection through to portfolio generation and assessment, and finally to producing explainable recommendations.

2. **Support personalised and suitable recommendations.**

Personal suitability should be at the forefront of both portfolio construction and analysis.

3. **Generate useful and explainable portfolio analysis.**

The system should calculate relevant portfolio metrics and present recommendations in a way that helps users understand the reasoning behind the output.

4. **Evaluate the system in a cloud-native environment.**

The system should be deployed and evaluated in a cloud environment selected through comparison of different architecture, infrastructure, and tooling options. The evaluation should measure this environment against the relevant functional and non-functional requirements.

2.5.3 Functional Requirements

The functional requirements define the core behaviours that the system must provide to meet the thesis objectives listed above. They define what the system must be able to do to be considered complete and primarily relate to the application's functionality rather than its technological performance.

ID	Module/Component	Description
FR-01	User Management	The system must allow users to create, view, and update an investment profile containing their preferences (goals, risk tolerance, budget, time horizon, constraints).
FR-02	Portfolio Management	The system must allow users to input and update current portfolio holdings to serve as context for recommendations.
FR-03	Data Engineering	The system shall ingest, process, and format market data into a form suitable for analysis, model training, and inference.
FR-04	Model Training	The system should support repeatable training, evaluation, and logging of machine learning models using historical or simulated market data.
FR-05	Model Registry	The system shall version and store trained models, experiment outputs, logs, and associated metadata so that model behaviour and experiment results can be reproduced and compared.
FR-06	Analytics Engine	The system shall compute relevant asset-level outputs and portfolio-wide metrics, including exposure, risk, concentration, and diversification.
FR-07	Agentic Decision logic	The system must include agentic decision logic capable of using user profile information and processed market data to generate and assess potential portfolio recommendations.
FR-08	Recommendation Engine	The system shall generate and rank portfolio recommendations, providing explanations for the key factors influencing each result.
FR-09	User Interface	The system must present the portfolio analysis, model metrics, ranked recommendations, and explanations via a user-facing interface.

Table 2.2: Functional Requirements

2.5.4 Non-Functional Requirements

The non-functional requirements define the main technical qualities that the system should satisfy and that will later be measured during evaluation. They do not all require fixed numerical thresholds at this stage. Section 6 further defines how each requirement will be evaluated, including the acceptance criteria where these can already be established and how any remaining operating limits will be determined.

ID	Category	Non-Functional Requirement
NFR-01	Latency	The system should generate portfolio analysis and recommendations within a response time suitable for interactive use.
NFR-02	Throughput	The system should be able to process multiple user requests, market data events, or recommendation tasks within a defined period.
NFR-03	Scalability	The system should be able to handle increased workload, such as more users, more assets, or more frequent market data updates.
NFR-04	Reliability	The system should either maintain core portfolio analysis and recommendation functionality or fail in a controlled and observable way during expected failures, restarts, or component interruptions.
NFR-05	Resource Usage	The system should use cloud resources efficiently, including CPU, memory, storage, and network usage.
NFR-06	Data Processing	The system should process market data quickly enough for recommendations to reflect current market state.
NFR-07	Observability	The system should provide enough logs and metrics to measure performance, identify bottlenecks, and support system evaluation.

Table 2.3: Non-Functional Requirements

2.5.5 Out of Scope

Everything not defined in Section 2.5.1 is considered to be out-of-scope for this thesis. However, there can be some ambiguity with specific areas. This section addresses this ambiguity by defining exactly what the areas are that are out of scope, which may be construed as being in scope.

Area	Not Included in Scope
Broker Integration and Trading	The system is a decision support tool. It does not connect to external brokers or trading platforms. It also does not execute investments for users.

Real-time data	The system is intended as an academic prototype, as such direct integration with paid or production-grade real-time market data feeds is out of scope. The system will instead use historical or recently sourced market data for development and evaluation.
Production and Regulated Financial Advice	The system is intended as an academic prototype and will not be developed as a regulated financial advice product. It will not be certified for production use, and the thesis will not address the full legal requirements for providing regulated financial advice.
Guaranteed Investment Performance	The system will not guarantee investment returns or claim that recommended portfolios will outperform the market. Recommendations are uncertain because future market behaviour cannot be known in advance.
Real Customer Data	The inputs into the system will use synthetic or user-entered data, rather than sensitive real customer datasets.
Institutional-Grade Portfolio Optimisation	The thesis will not implement complex hedge-fund or institutional-grade portfolio optimisation methods.

Table 2.4: Out-of-Scope System Areas

2.5.6 Success Criteria

For the thesis system to be considered successful, it must satisfy the functional requirements and demonstrate the technical behaviour defined by the non-functional requirements. This means that the system should provide a complete portfolio decision-support workflow, from user profile and portfolio input through to market-data processing, model training, agentic decision logic, recommendation generation and user-facing explanation.

Primary success will be determined by two main factors:

- Whether the system can generate ranked, personalised recommendations that align with the user's investment profile, preferences, and constraints.
- Whether this can be done efficiently in a cloud-native environment, as assessed against the non-functional requirements.

The real investment performance of the portfolios and the system's production readiness are both considered secondary to the two factors listed above.

2.6 Inputs and Outputs

This section defines the main information that the system is expected to receive, use, and produce. The inputs include both user-provided information and externally provided information. The outputs include the user-facing recommendations, but also the intermediate system artefacts, and evaluation metrics.

2.6.1 User Inputs

User inputs are obtained directly from the user through the interface. These inputs allow the system to provide personalised recommendations rather than producing the same output for every user. They can be grouped into preferences, constraints and current holdings. Preferences describe the user's investment goals and preferred portfolio characteristics such as their risk tolerance and investment goals. Constraints define limits that recommendations should adhere to such as their budget. And current portfolio holdings provide the information relating to the user's existing investment position.

2.6.2 System Inputs and Configuration

System inputs are the information required by the system that is not manually entered by the user. Market data is the main external data input, providing the information required to generate and asset portfolios. Additionally, external information about the configuration of the system must be provided. This data will define how the system, models, deployment environment, and evaluation process should operate. This data supports reproducibility by ensuring that model behaviour, system behaviour, and experiment results can be traced back to the inputs and settings used to produce them.

2.6.3 Intermediate, User facing and Evaluation Outputs

For this system the outputs should be threefold, comprising of intermediate outputs, final user-facing outputs, and evaluation outputs. The intermediate outputs are independent service or stage outputs that support the later analysis. These include aspects such as processed market data, processed user request data, and generated portfolios. The final user-facing outputs are everything that is provided to the user after their inputs are processed. This is everything comprising the ranked portfolio recommendations and supporting explanations. The system should also produce evaluation outputs. These are used to assess the technical behaviour of the system against the non-functional requirements and support the greater performance analysis.

2.7 Specification Summary

This system specification defines the intended purpose, scope, requirements, inputs, and outputs of the proposed portfolio decision-support system. The system is expected to collect user profile information, and relevant market data in order to produce and assess personalised portfolios for individual users. The system will provide an end-to-end decision-support workflow, with the exact internal design of this workflow defined in the design plan (Section 5). The system will be considered successful if it can generate appropriate and explainable recommendations that align with the user's profile and constraints. It should also be able to do this efficiently in a cloud-native environment, with measurable performance and appropriate use of computing resources.

3. Literature Review

This literature review identifies the concepts required to support the design and evaluation of a cloud-native agentic AI portfolio decision-support system. It is not intended to provide a full review of finance, artificial intelligence or cloud computing. Instead, it focuses on the areas that directly influence the later design and evaluation plans: portfolio decision support, investor profiling, financial prediction models, model management through Machine Learning Operations, agentic AI, cloud deployment architectures, and performance testing.

The purpose of this section is to establish the research basis for later decisions. It therefore evaluates relevant ideas from the literature but does not make final design choices. Those choices are made later in the report, using both the literature and the project experiments.

3.1 Portfolio Decision Support

This section reviews the basic investment and portfolio-management concepts that are relevant to financial decision support. It first explains portfolios, risk, return and diversification. It then distinguishes between active and passive investment strategies, as the type of strategy affects the form of support an investor requires. Finally, it discusses robo-advisors and financial decision-support systems as examples of software systems that translate investment information into user-facing portfolio guidance.

3.1.1 Portfolio Management Principles

Investing involves allocating capital to assets with the expectation of receiving future returns. These returns may come from capital gains, dividends, interest payments or other forms of income. The proposed system is most closely related to stock-based portfolio decisions, although portfolio-management concepts are not limited to stocks. Investors can also hold bonds, funds, ETFs, real estate and other assets. The important point is that investment involves uncertainty. An asset may increase in value, but it may also fall, produce lower returns than expected, or expose the investor to losses.

The central investment trade-off is between return and risk. Return refers to the gain or loss produced by an investment. It may be realised, which means it has already occurred, or expected, which means it is estimated before the investment decision is made. Risk refers to the uncertainty of that return. In portfolio literature, this is often related to volatility, which describes how much an asset's price or return tends to fluctuate. A more volatile asset can produce higher gains, but it can also produce larger losses. Lower-risk assets, such as government bonds, tend to have lower expected returns because their cash flows and default risks are usually more predictable [1].

A portfolio is a collection of assets, with each asset representing a proportion of the total investment. The main advantage of a portfolio is diversification. Diversification means holding assets that are not perfectly correlated, so they do not all respond to market events in the same way. This reduces dependence on the performance of a single company, sector or asset, which in portfolio theory, is known as reducing unsystematic risk. Portfolio risk is comprised of unsystematic risk, which is specific to individual assets, and systematic risk, which affects the broader market. Diversification reduces this unsystematic risk[1] which is the main benefit of holding a balanced portfolio: the portfolio risk is lower than the average weighted individual risks of all the assets. This idea is relevant to portfolio decision support because a recommendation should not be assessed only by the predicted return and risk of

[1] But cannot reduce systematic risk, as this is market wide.

an added asset. But it should also be assessed by how the asset changes the overall composition and characteristics[2] of the portfolio [1], [2].

Portfolio management literature treats the risk-return relationship as a central part of investment decision-making. Stewart, Piros and Heisler discuss client objectives, risk definitions and diversified portfolio construction as part of the portfolio management process [2]. Fabozzi and Markowitz also frame investment management around asset allocation, valuation, portfolio construction and investment strategy [1]. These sources are useful because they show that portfolio decisions should be based heavily on selecting attractive assets with regards to risk, return and diversification. However, they also introduce the idea of selecting assets that fit the investor's objectives, constraints and risk tolerance as well – a topic which is further expanded on in Section 3.2.

3.1.2 Active and Passive Investment Strategies

Investors are often described as being either passive or active with regards to their investment strategy. Passive investing involves holding a diversified portfolio with the aim of matching a benchmark or maintaining a target allocation over time. Active investing involves more frequent analysis and more selective adjustment of holdings in an attempt to outperform a benchmark or respond to market opportunities [2].

The distinction matters because each strategy requires different support. Passive investing is less concerned with continuous market prediction and more concerned with whether the portfolio still matches its intended composition or initial characteristics. If one asset rises faster than the others, the portfolio may drift away from its target weights – changing the state of the portfolio. Rebalancing in a process which is used to bring the portfolio back towards its intended state. In this context, software support is mainly used to monitor portfolio drift, compare the current portfolio against the target, and identify when rebalancing may be appropriate.

Active investing requires a different type of support. An active investor may need more frequent analysis of market data, model outputs, asset opportunities and portfolio consequences. This does not necessarily mean day trading. It means that portfolio changes are based on analysis, forecasts or perceived opportunities rather than only on maintaining a predetermined composition or characteristics set. Literature on robo-advisors and digital wealth management is relevant here because it shows that investment software can support both long-term allocation management and more frequent portfolio review [3]. The key research point is that portfolio support should be interpreted in relation to the user's investment strategy rather than as a single universal recommendation process.

3.1.3 Robo-Advisors and Financial Decision-Support Systems

A financial decision-support system helps users make investment decisions by collecting relevant information and presenting analysis and recommendations. In portfolio management, this may involve combining market data, current holdings, user preferences, risk measures and portfolio metrics. The system does not need to execute trades to be useful. It can support the user's decision by translating complex financial information into a more understandable form.

Robo-advisors provide a useful example of this type of support and provide a base reference for systems that can take user information and translate it into investment recommendations. Sironi

[2] In this report composition refers to the portfolio’s assets and their weightings; whereas characteristics refer to the portfolios resulting properties, such as its risk, expected return and diversification.

introduces the idea that user preferences can be turned into signals and mapped to portfolio profiles through robo-advisor systems [3]. He talks about the importance of personalisation when constructing portfolios for users and introduces robo-advisory systems as a technique for this.

However, robo-advisors are usually more structured and narrower than the agentic decision-support system considered in this report. Many robo-advisors use questionnaires, risk scores and pre-defined model portfolios [3]. This creates a useful baseline: investor information is collected, translated into a profile containing signals, and connected to a portfolio. A more advanced financial decision-support system, however, would extend this by combining user profiles with constructed portfolios through various dedicated AI models. The literature therefore supports the idea of personalised portfolio and can be used as a baseline on how to achieve this in a simplified manner.

3.2 Investor Profiling and Portfolio Suitability

This section reviews how investor information is collected, organised and compared with investment options. Suitability is important because there is no universal portfolio that fits every investor. Different investors have different goals, time horizons, financial situations, risk preferences and constraints. This section first explains investor profiles, then separates profile information into objectives and constraints. Finally, it discusses suitability frameworks and portfolio metrics, including how user preferences and asset or portfolio characteristics can be compared during suitability assessments.

3.2.1 Investor Profile Information and Suitability Frameworks

Investor profile information refers to the personal and financial information used to understand which investment strategy or portfolio may be appropriate for a user. It is not limited to a simple risk score. A useful investor profile may include investment goals, current holdings, income or financial situation, investment experience, time horizon, liquidity needs, risk tolerance and ability to bear losses [4], [5]. This information matters because two investors can have the same desired return but still require different portfolios if their time horizons, existing holdings or loss tolerance differ.

Regulatory suitability frameworks provide a strong basis for identifying the types of information that should be considered, as they disclose the information that must be collected by financial professionals to protect investors. The idea is that a financial professional must have sufficient information about the investor to provide adequate and properly informed recommendations. FINRA Rule 2111 is a US suitability rule, and states that a customer investment profile must include: age, other investments, financial situation and needs, tax status, investment objectives, investment experience, investment time horizon, liquidity needs, risk tolerance and other relevant information disclosed by the customer [4]. MiFID II Article 25 is the EU equivalent, and states that financial professionals must collect information about the user pertaining to their knowledge and experience, financial situation, and investment objectives [5].

These frameworks are not system-design documents, but they are useful because they show that suitability requires a broad view of the investor. A recommendation cannot be assessed only by whether an investor says they want high returns. Other factors such as the user's experience, ability to understand risk, ability to bear losses and need for liquidity also affect whether a recommendation is appropriate. This is relevant to later design and evaluation planning because investor profiles can be represented as test cases. The system can then be assessed based on whether it changes its recommendations when investor objectives, constraints or risk capacity change.

3.2.2 Investment Objectives and Constraints

Investor profile information can be organised into objectives and constraints. Objectives describe what the investor is trying to achieve. These may include capital growth, income generation, capital preservation or a target level of return. Risk tolerance is also connected to objectives, because investors usually seek a particular balance between expected return and acceptable uncertainty. Constraints, on the other hand, describe the limits within which those objectives must be pursued. These may include time horizon, liquidity needs, tax status, legal or regulatory restrictions, ethical exclusions, sector restrictions, investment budget or concentration surrounding an existing portfolio. This distinction is backed up by the CFA Institute's portfolio planning material which provides a clear structure for the distinction through the investment policy statement [6]. This structure is useful because it separates what the investor wants from the limits that shape what is suitable.

The distinction also matters for automated support. A preference may guide the recommendation process, while a constraint may restrict it. For example, a user may prefer technology exposure but also require low portfolio concentration. Another user may want long-term growth but need enough liquidity to access funds within a short period. In both cases, the system needs to compare the portfolio against more than one type of information. Objectives guide the desired direction of the portfolio, while constraints define boundaries that should not be ignored.

3.2.3 Suitability Assessment

Suitability assessment compares investor characteristics with investment or portfolio characteristics to determine whether a recommendation is appropriate. In general, the process begins by collecting information about the investor. The assets or portfolios being considered are then assessed against that profile. MiFID II Article 25 also requires suitability reports to explain how advice or portfolio management meets the client's preferences, objectives and other characteristics [5]. This shows that suitability is not only a calculation, but it also involves explaining why the recommendation fits the investor. This is important to note for the design of the systems output.

Robo-advisor literature provides a practical example of suitability assessment in automated investment systems. Sironi's discussion of goal-based investing, risk profiling and model portfolio construction shows how user responses can be converted into risk scores and then matched to model portfolios [3]. Suitability assessment can therefore be understood as a matching process. On one side is the investor profile, and on the other side is the investment or portfolio, both translated into their relevant signals. A recommendation may appear attractive based on expected return or model prediction but still be unsuitable if it violates the user’s constraints or doesn’t match their objectives.

3.2.4 Portfolio Metrics and Derived Suitability Signals

Many portfolio suitability measures are not entered directly by the user. They are calculated from holdings and market data. CFA portfolio mathematics explains that expected portfolio return is a weighted average of the expected returns of the securities in the portfolio, whereas the portfolio variance depends on individual holdings and relationships between returns, and is typically lower than the average weighted risk of the assets [6]. The portfolio's risk and return characteristics are, therefore, derived from the assets the portfolio contains, their weights and how these assets move relative to one another.

Derived sector metrics or signals can include allocation, concentration, volatility, beta, correlation, covariance, sector exposure and diversification to name a few [6]. An everyday user may not be able to define these technical values, but they can still provide meaningful information about goals, risk tolerance and constraints. The system or analyst then calculates the portfolio-level metrics needed to assess suitability relative to the user’s preferences. This distinction is important because it separates

user-facing information from technical portfolio analysis: despite not producing the same signals, user profiles and portfolio characteristics can still be compared for suitability.

Suitability assessment links the two defining a suitable portfolio as one where the portfolio-side signals align closely enough with the investor's requirements. This calculation is done through a mapping or comparison software. This software would recognise, for example, that a profile of a more conservative user profile should not match well with a portfolio that has high volatility, high concentration and poor diversification.

3.3 Machine Learning Lifecycle for Financial Prediction

This section recognises the extended lifecycle of model creation and management of that lifecycle through Machine Learning Operations (MLOps) methodologies and tools. The section first provides an overview of the end-to-end process, and then individually goes into each stage of this lifecycle. This is important for all tools in the system that use machine learning models and ensuring that these tools are created in an optimal way and managed correctly after deployment.

3.3.1 Lifecycle Overview

The machine-learning lifecycle begins before a model is trained. A model first requires data to be collected, cleaned and transformed into a suitable format. The prediction task then needs to be defined, including whether the model predicts a category, such as price direction, or a continuous value, such as future return. The model is then trained, tuned and evaluated. If it performs well enough, it may be registered and deployed. Once deployed, it must be monitored because incoming data and model behaviour can change over time [7], [8].

The notable part of the ML lifecycle is that the direction is not strict. At any point in the lifecycle the programmer can go back one or multiple steps. If, for example, the model is being trained and the features are identified as the problem causing reduced performance, then the programmer should go back and derive different features to represent the data. The requirements of the model should always be of the highest importance, if these are not being met, it is critical that it is evaluated as to why this is. Steps, such as backtracking, should then be made to correct these issues.

MLOps literature describes the practices and tools used to manage this lifecycle, helping to ensure backtracking is possible, and providing ways to approach and solve any issues. Treveil et al. describe MLOps as a way to coordinate data preparation, model development, deployment and monitoring into one cohesive manageable process [7]. Huyen also emphasises that deployment is not the end of machine learning work, and that model’s still need to be continually monitored after deployment [8]. Production monitoring can reveal changes in data distributions or model performance, which may require retraining or another development iteration. This is particularly important for financial models because markets change over time and past patterns may not continue under new conditions.

This lifecycle perspective is relevant because it prevents the model from being treated as a static artefact. A prediction model used in a decision-support system needs traceable data, repeatable training, meaningful validation and monitoring after deployment. Tools can support each part of this process providing ways to, for example, store models and track experiments.

3.3.2 Financial Market Data and Sourcing

Financial prediction models rely on market data. The most common data type is price data, which can range from high-frequency tick or order-book data to intraday and daily aggregates. Tick data can provide a detailed view of market activity, but it is noisy and requires more storage, preprocessing and

computational resources. Aggregated data is often represented as OHLCV data, which records the open, high, low, close and volume for a given period. Jansen discusses these market data formats in the context of machine learning for trading, noting the importance of the frequency of data [9].

Financial models can also use fundamental data. Fundamental data describes the economic characteristics of a company, such as revenue, earnings, assets, liabilities and cash flows [10]. This information is usually sourced from company reports or regulatory disclosures, including structured filings such as XBRL data. Alternative data provides another category and may include financial news, earnings-call transcripts, social media, web data or sentiment indicators. Jansen and Tatsat et al. discuss these data types as possible inputs for financial machine-learning systems, although alternative data often requires more complex processing and may raise additional quality concerns [9], [10].

Data frequency and source quality affect model suitability. Higher-frequency data provides more observations and more detail about short-term market behaviour, but it also contains more noise and increases processing requirements. Lower-frequency data is easier to work with and may be more appropriate for longer-term portfolio support, but it loses information about intraday movements [9]. Public or low-cost services such as yfinance can provide historical data from Yahoo Finance, while detailed real-time feeds, such as Nasdaq TotalView-ITCH, provide tick-by-tick market events and are usually more complex and costly [9]. The choice of data source therefore affects not only model training, but also deployment cost, latency and system complexity.

3.3.3 Feature Engineering, Labelling and Validation

Raw financial data usually needs to be transformed before it can be used effectively by a model. Feature engineering creates input variables that represent useful information from the raw data. Common financial features include moving averages or returns, volatility measures, momentum indicators and lagged observations. These are often calculated using rolling windows, which summarise recent market behaviour over a defined period. Jansen discusses financial feature engineering as a way of creating alpha factors, which simply put are the mathematical signals used to predict future price movements [9].[3]

Labelling defines what the model is trying to predict. In a classification task, the label might describe whether an asset's price increases or decreases over a future horizon. In a regression task, the label might be a future return value. Tatsat et al. describe both classification and regression as common supervised-learning formulations in finance, with the purpose of the model dictating which type to use [10]. The distinction between task types is important as the label should be chosen relative to what the model is trying to achieve. A price-direction classification model may be evaluated using accuracy or F1 score, while a return-prediction regression model may require error metrics such as mean absolute error or root mean squared error.

Validation is especially important for financial time series because observations are chronologically ordered. To prevent data leakage and ensure the model has no access to future information, training must precede the target evaluation periods. Cross validation with a rolling window[4] addresses this by training on earlier data and then validating and testing on later data [11]. The training, validation and testing windows are then moved forward through time iteratively until the model has been trained on all the data. This better reflects the real forecasting problem. The central point is that model evaluation should preserve the chronological structure of financial data. Otherwise, the model may appear more

[3] Alpha actually refers to the excess return above a market benchmark

[4] Also known as walk-forward validation

accurate than it would be in deployment as it would have effectively been given the answers, rather than actually learning he predictive patterns.

3.3.4 Prediction Models and Their Limitations

Several types of models can be used for financial prediction. Simple baseline models, such as linear regression for continuous predictions or logistic regression for directional classification should be built as a common starting point. These models provide a benchmark which can be expanded on to meet the specific requirements of the application. If more complex models cannot perform better than the simple baseline, then the added complexity may not be justified [9], [10].

Tree-based models are a common extension because they can capture nonlinear relationships between features. Random forests combine multiple decision trees trained on different samples of observations and features, which can reduce the instability of individual trees. Gradient-boosting models, such as XGBoost and LightGBM, train weak learners sequentially so each new learner focuses on reducing the errors of the previous ones. These models are widely used in machine learning for sequential tasks by framing the historical data as independent tabular features [9].

Traditional time-series models can also be useful. Autoregressive models use windows of past observations to forecast future values. Autoregressive Integrated Moving Average (ARIMA) models are commonly used for time-series forecasting, while Generalised Autoregressive Conditional Heteroskedasticity (GARCH) models are commonly used for volatility forecasting. The main limitations of these models are that their pattern recognition is limited to the size of the window. Neural sequence models, such as Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) can also be used. They instead use hidden state vectors that represent history. LSTM models are an extension of RNNs that better captures long-term patterns [11].

Financial prediction also has unavoidable limitations. Market data is noisy, relationships change over time, and historical patterns may not generalise to future conditions. This non-stationarity makes overfitting a serious risk. A model may learn patterns that existed in one market period but fail when the regime changes. Poor validation or leakage can make this worse by producing overly optimistic results. Backtesting can help estimate how a model or strategy would have performed historically, but backtests depend on assumptions about transaction costs, liquidity, timing and execution. Strong historical performance therefore does not guarantee future investment usefulness [9], [10], [11].

This is relevant to decision support because model output should be treated as a signal rather than a final decision. A good practice is to compute a confidence score for each metric produced. Due to these limitations, it is also strategic, from an accuracy perspective, to have individual dedicated models for each metric, rather than one model to produce all signals. This would significantly increase complexity though which should also be taken into account.

3.3.5 Experiment Tracking and Model Registry

Developing machine-learning models differs from developing ordinary software. Software will either run correctly, or there's an error. Machine-learning models, on the other hand, can run without errors and still have a poor performance. Working code is less about having no bugs, and more about correct model choice, features, and hyperparameter tuning. Experiment tracking records the configuration and results of each training run so that model performance can be compared across iterations. This can include datasets, features, labels, hyperparameters, training code, metrics and model artefacts. Huyen discusses experiment tracking as part of reproducible model development, in conjunction with model and data versioning [8].

A model registry stores trained model versions and metadata. It provides a controlled link between model development and deployment. MLflow is a practical example of a tool that supports experiment tracking and model registry workflows [12]. The current MLflow documentation describes tracking around runs, which are executions of data-science code, and describes the model registry as a mechanism for versioning and managing models through a UI and API. This is relevant because a deployed system needs to know which model version produced a prediction, especially when predictions influence later portfolio recommendations.

Model registries also support reproducibility. If a model performs unexpectedly, the training data, feature set, parameters and metrics should be traceable. This is particularly important in financial systems because model behaviour may change after retraining or when market conditions shift. Without model lineage, it becomes difficult to explain why one recommendation was produced by one model version and a different recommendation was produced later by another.

3.3.6 Monitoring, Drift and Retraining

Once a model is deployed, it must still be monitored. Drift occurs when the data or relationships seen in production differ from those seen during training. Covariate drift refers to a change in the distribution of input features. Label shift refers to a change in the distribution of outputs. Concept drift refers to a change in the relationship between inputs and outputs. Huyen and Crowe et al. discuss these forms of drift as reasons why production monitoring is needed after deployment and note the changes that need to be made to absolve these drifts when retraining [8], [13].

Financial models are especially exposed to drift because market behaviour changes due to a variety of factors such as interest-rate changes or recessions. The system should therefore monitor input distributions, prediction distributions and model predictive performance where labels are available for all models used. There should also be a tool that monitors recommendation suitability. When any monitor detects significant degradation, the model that is being monitored may need to be retrained using newer or more representative data.

Retraining should still follow the model-development lifecycle. A retrained model should be evaluated, tracked and versioned before replacing the deployed model. This supports reproducibility and reduces the risk of replacing a known model with an untested one. It also links model management to the later evaluation plan, because model quality should be assessed not only at training time, but also during operation.

3.4 Agentic AI

This section discusses agentic AI as a way of coordinating the information and tools required for complex decision-support tasks. Traditional machine-learning models generally produce predictions from input data. Agentic systems extend this by reasoning through multiple steps, calling tools, observing results and adapting the next step of the process. This is relevant to the thesis as the system design needs to autonomously create a suitable portfolio and reason through its decisions to do so. This section first covers the main concepts about agentic AI, then it discusses single- and multi-agent architectures. Finally, it talks about guardrail architectures, the importance of explainability, and the evaluation of agentic systems.

3.4.1 Agentic AI Concepts and Tool Use

Agentic AI refers to systems that use language models or other AI components to perform multi-step tasks through reasoning, tool use and interaction with an external environment. ReAct is a useful

foundational source that describes how agentic systems work through a thought, action, and observation loop. Yao et al. wrote a breakthrough paper in 2022 which shows that a model can alternate between reasoning about a task, taking an action, observing the result and continuing the process without human intervention [14]. This is different from a standard static prediction model because the system can decide what information it needs next.

This concept fits decision-support tasks because the relevant information may sit across several tools. A portfolio-support workflow may need a user-profiling model, an asset-signal model, a portfolio construction algorithm, a correlation calculation, a diversification score, a risk calculation and a final explanation step. The agent does not replace these tools. It coordinates them. This makes the agentic layer useful when the task is not simply to predict one value, but to search, evaluate and revise possible outputs until a suitable result is found.

AutoGen extends this idea by showing how multiple agents can cooperate through conversation while using tools and human input [15]. This supports the broader idea that agentic systems can be structured either as one tool-using agent or as several specialised agents. The key literature point is that agentic AI provides an orchestration layer over tools, data and models. This is a key idea for the thesis design as it should be reasoned why a single agent, or multiple agents would better suit the requirements of the task.

3.4.2 Agentic Architectures

Multi-agent architecture naturally aligns with portfolio decision support, where the workflow can be decoupled into distinct phases: portfolio generation and portfolio assessment. Multi-agent literature demonstrates that decomposing a complex problem into highly specialised, role-based workflows drastically reduces the cognitive load on a single Large Language Model (LLM) prompt while introducing structural checkpoints [15].

This role-based separation is common in financial agent literature to help manage complex tasks. For example, market frameworks like *FinRobot* and *TradingAgents* show that financial reasoning works best when split among specialized roles, such as researchers, analysts, or risk managers [16], [17]. Even though these platforms focus on active market trading rather than personal portfolio design, their core lesson still applies: splitting a complex financial task into separate roles makes it easier to manage.

For portfolio decision support, the literature supports two broad agentic architectures. One architecture optimises the use of single centralised agent that uses several tools to generate and iteratively refine portfolios until one or multiple ranked portfolios are found that satisfy the relevant requirements [14]. Another pattern is a two-agent loop where one agent proposes portfolios, and another assesses them against risk, diversification, constraints and suitability [18]. The research does not determine which structure is best, but it provides a basis for evaluating both. In either case, the agentic process should be judged by whether it uses the correct information, respects constraints, and produces portfolios that align with user and portfolio signals.

3.4.3 Guardrails and Explainability

Agentic systems require guardrails because they can take multiple steps and may combine information from several tools. Guardrails are a safety and validation layer that surrounds LLMs and controls the inputs and outputs [18]. In financial decision support, this matters because a misleading output could cause inappropriate user decisions. Explainability is therefore not just about making the final recommendation readable. It involves showing what information was considered, what calculations or tools were used, and why the result is suitable or unsuitable. This helps with evaluation of the system,

as if it can explain why decisions were made, this helps to infer whether the recommendations are relevant to the user. It also helps to decipher at which stage any potential errors may have occurred.

GuardAgent is relevant because it proposes a separate guard agent that checks whether a target agent's inputs and outputs satisfy specified requirements [18]. This concept maps closely to suitability assessment and the proposed two-agent system discussed in Section 3.4.2. A guard or assessment component can check whether a proposed portfolio violates a risk constraint, exceeds a budget, creates unacceptable concentration, fails a liquidity requirement or lacks sufficient explanation.

3.4.4 Evaluation of Agentic Systems

Evaluation is also different for agentic systems than for ordinary prediction models. Tau-bench is a method that evaluates tool-using agents based on task completion, tool interaction, rule-following and consistency across repeated trials [19]. This is relevant because a portfolio agent should not only produce a plausible answer once. It should behave consistently across similar user profiles and market scenarios. Agentic evaluation therefore needs to consider task success, constraint compliance, repeatability, explanation quality and whether the right tools were used. This literature is relevant to the later evaluation plan because it suggests that agentic systems should be tested through fixed scenarios rather than judged only by the apparent quality of their generated text.

3.5 Cloud Deployment Architectures

This section reviews cloud deployment architectures relevant to hosting and evaluating the proposed system. The project is concerned with how different deployment approaches suit different parts of the overall system, particularly with regard to performance, scalability and resource usage. This section delves in to these primary approaches and the trade-offs associated with them.

3.5.1 Cloud-Native Systems

Cloud-native systems are designed to take advantage of cloud environments rather than simply being placed on remote servers. The Cloud Native Computing Foundation (CNCF) defines cloud-native practices as supporting scalable, repeatable and programmatic deployment across public, private and hybrid environments through the use of technologies such as containers, microservices, immutable infrastructure and declarative APIs [20]. They also describe cloud-native approaches as an enabler for loose coupling, which in turn creates resilient, manageable and observable systems.

These properties are relevant because the system being studied contains different workload types with different technical requirements. User-facing recommendation requests are likely to be more latency-sensitive, while market-data processing may involve larger background workloads that are more concerned with throughput and resource use. Different cloud-native architectures provide different ways to deploy these workloads, allowing each part of the system to be monitored, updated and scaled in ways that are suited towards their specific requirements. However, these advantages come with trade-offs, as each architectural approach affects performance, resource usage and operational complexity differently.

3.5.2 Monolithic and Microservice Architectures

Monolithic and microservice architectures describes two main ways that an application can be deployed. A monolithic architecture packages the main application as one singular unit that handles everything. Separate modules may exist inside the codebase, but they are deployed together and communicate through in-process calls. This can make development and testing simpler, especially

while the system is still changing. However, it also introduces limitations such as being a single point of failure and having limiting scaling capabilities [21].

A microservice architecture decomposes the application into smaller parts that can be deployed independently. Dragoni et al. describe microservices as an architectural style where applications are structured as sets of small services that communicate with one another over a network [21]. In this context, services could be separated around different workload areas, such as user-request handling, market-data processing, portfolio generation, and recommendation evaluation. This can improve independent scaling and maintainability of each service, while also separating issues so that there is a smaller effect on the entire system when failures occur.

The literature also shows that decomposition has costs. Faustino et al. analyse a stepwise migration from a monolith to microservices and show that modularisation and inter-service communication can introduce significant performance overhead [22]. Gan et al.'s DeathStarBench – an end-to-end microservices benchmark suite – work also shows that microservice applications introduce critical performance bottlenecks across networking, cluster management, quality of service and tail latency [23]. The Istio case study shows that when microservices become too complex to manage, consolidating them back into a single system is often the better choice [24]. This research supports the idea that microservices are not automatically better and that their suitability depends on workload and requirements. The benefits associated with the microservices approach can easily become outweighed by their drawbacks as complexity increases.

3.5.3 Event-Driven and Serverless Architectures

Event-driven architecture describes a communication pattern where components produce and consume events. This differs from synchronous request-response communication, where one service calls another and waits for a result. In this system, event-driven communication is most relevant to background market-data processing, where updates can be handled separately from user-facing requests. A market-data component could publish updates as events, while downstream components could consume and distribute these events for processing. To implement this pattern, platforms like Kafka [25] act as event streams to route messages between producers and consumers, while specifications like CloudEvents [26] provide a standardized format for structuring event data across services [27].

Event-driven systems also reduce direct coupling because producers do not need to know which downstream consumers will process an event. This is useful for data streams and background workloads, where processing demand may vary and additional consumers can be added if required. However, event-driven systems also introduce additional complexity. Laigner et al. identify practical challenges in event-based microservices relating to event schemas, payload design, event ordering, auditing and monitoring [28]. The event-driven architecture can therefore improve decoupling while making debugging and traceability harder.

Serverless computing represents a complementary execution model centred on infrastructure abstraction. Jonas et al. describe serverless computing as a paradigm where cloud management and resource provisioning are fully automated, which reduces the need for developers to manage servers directly [29]. Serverless workloads are often triggered by requests or events and can scale dynamically in response to incoming demand. Knative provides a Kubernetes-based serverless model that supports autoscaling and scale-to-zero capabilities [30].

The primary trade-off of serverless architectures is between resource efficiency and latency predictability. Golec et al. identify cold-start latency as the key challenge of serverless computing. They

mention that while scale-to-zero reduces idle resource usage, it will likely also increase response time when a service needs to start again [31]. Djemame and Norton's Knative evaluation supports a similar view, showing that while scale-to-zero can reduce idle power consumption, overall performance depends on the workload and application type [30]. This trade-off is relevant to the proposed system because some supporting tools may only be used occasionally, while others may need to respond quickly as part of a user-facing request. Serverless deployment should therefore be evaluated based on the access pattern and latency requirements of the specific component rather than assumed to be suitable for every part of the system.

3.5.4 OpenStack and Kubernetes

Cloud computing services are often provisioned based on how much control the user needs. OpenStack operates at the Infrastructure-as-a-Service layer, providing compute, storage, and software-defined networking through standardised APIs [32]. This gives the user more control over the virtual infrastructure but also means that they must manage more of the deployment environment. Kubernetes operates above this layer as a container orchestration platform and is commonly used with IaaS platforms such as OpenStack [33]. In simple terms, OpenStack provides the virtual servers, while Kubernetes manages the containerised applications running on them.

This combination is relevant to architecture evaluation because it allows different deployment approaches to be tested while keeping the underlying resources controlled. OpenStack can provide the virtual infrastructure, while Kubernetes can manage deployments, replicas, scaling behaviour, and CPU and memory limits. This matters because comparisons between monolithic, microservice, event-driven, serverless, or workload-placement approaches should reflect architectural trade-offs rather than unequal access to compute resources [34]. This is also why the later experiments keep application logic, datasets, and resource limits consistent where possible when comparing different deployment approaches.

3.5.5 Architecture Trade-Offs

Architecture choices always involve trade-offs meaning that there is no ideal deployment style for every part of the system. A monolithic approach is useful when latency is the highest priority as different stages of the application communicate inside one container rather than through several network calls. The drawback is that it becomes harder to scale, change, or isolate one part of the system without affecting the rest. Microservices solves some of this by splitting the application into smaller services with clearer boundaries. This can make individual parts easier to manage, but it also introduces significant communication overhead as requests need to move between services located in different containers [21]- [24].

Event-driven and serverless architectures are more suited to specific workload patterns. Event-driven processing can be useful for background updates as work can be queued and handled without affecting the main data stream. However, event-driven approaches also makes tracing and debugging harder. Serverless deployment has a similar trade-off in that it can reduce idle resource usage for tools or services that are not used often, but the approach introduces significant cold-start delays and has less predictable response times [28], [29], [30].

These trade-offs provide the basis for the later architecture evaluation. Different workloads should be evaluated against the requirements that matter most to them, rather than using one metric for the whole system. For example, a user-facing request path is more sensitive to response time and p95 latency, while market-data processing may be more concerned with throughput, queue behaviour, and resource usage. To make these comparisons meaningful, the underlying logic, data inputs, model

behaviour, and resource limits should remain as consistent as possible. This helps ensure that the measured differences come from the architectural approach being tested, rather than unrelated implementation differences.

3.6 Monitoring and Performance Testing

Monitoring and performance testing are important for both the architecture experiments in this project and the later evaluation of the completed system. The main concern is not simply whether the system works, but how well it behaves as the workload changes and where any performance limitations come from. This requires a combination of measures such as latency, throughput and resource usage, alongside a testing approach that keeps the surrounding conditions controlled enough for meaningful comparisons to be made.

3.6.1 Systems Performance and Observability

Systems performance describes how an application behaves while it is actually doing work, including how quickly that work is completed and where delays or resource limitations begin to appear. Gregg describes performance analysis as a combination of measurement, observability, experimentation, benchmarking and bottleneck analysis [35]. Observability specifically is the process of exposing what is happening inside the system through information such as metrics, logs and traces.

This becomes especially relevant once an application is split across several services. An increase in end-to-end latency may be easy to identify, but that alone gives little indication of where the extra delay came from. Depending on the architecture, the cause could sit within model processing, storage access, service communication, an event queue or even the activation of a serverless component. The SRE literature similarly views observability as a way of understanding behaviour across the different components of a distributed system rather than relying only on the final output [36].

Tools such as OpenTelemetry, Prometheus and Grafana provide practical ways of collecting this information. OpenTelemetry captures distributed traces across service boundaries [36], while Prometheus stores and Grafana visualises system-level metrics over time [35]. The exact tools are less important than having enough visibility to connect changes in overall performance back to what individual parts of the application were doing at the time.

3.6.2 Performance Metrics

There is no single metric that gives a complete picture of system performance. Latency and throughput show how quickly useful work is being completed, while CPU and memory provide some indication of the resource cost required to achieve that performance. Error rates add another dimension by showing whether the system can continue handling the workload as demand increases [35].

Latency also needs to be considered as a distribution rather than only as an average. A system may perform well for most requests while still producing a smaller number of very slow responses, which can easily be hidden by a single mean value. Percentiles such as p50 and p95 make this behaviour more visible by separating typical response times from the slower end of the distribution [35]. This is particularly relevant for distributed architectures, where service communication, queueing or serverless cold starts can affect some requests far more than others.

Other metrics become useful depending on what is being tested. Queue depth and signal staleness, for example, can reveal whether an event-driven pipeline is keeping up with incoming work even before errors begin to appear, while scaling speedup shows whether adding more processing instances actually results in a proportional increase in capacity. Resource use can also be viewed over time rather

than as a single utilisation value. This is important for intermittently used services, where an architecture may consume more resources while starting but considerably less while sitting idle.

3.6.3 Workload, Scalability and Reliability Testing

Performance results only have meaning in relation to the workload being applied. Request-based applications may behave differently as concurrency increases, while a data-processing pipeline may instead be affected by the amount of incoming data or the computation required for each item. Testing several workload levels helps show whether an architectural difference remains consistent or whether it only becomes important under particular conditions. This is more useful than evaluating a single workload and assuming the same behaviour will continue as demand changes [35].

Scalability testing looks at a related question: whether additional resources actually translate into additional processing capacity. In horizontally scaled systems this can be tested by increasing the number of service instances or workers and comparing the resulting throughput against a baseline. Perfectly linear scaling is unlikely because communication, coordination and other shared overheads remain, so the important part is understanding how quickly these additional resources stop providing the same return [35].

Reliability covers how the system behaves when normal operation is interrupted. This is less central to the architecture experiments conducted in this project, but it will still be relevant when evaluating the completed system. A failure should therefore be considered not only in terms of whether requests stop working, but also how much functionality is affected, how quickly the system recovers and whether normal behaviour returns once the failed component becomes available again [36].

3.6.4 Evaluation Methodology

A fair comparison depends on keeping unrelated factors as consistent as possible. Jain's work on computer-systems performance analysis places particular emphasis on choosing appropriate workloads, metrics and experimental factors before testing begins. When two architectures are compared, differences in application behaviour, input data, resource availability or deployment conditions can otherwise become mixed into the result. If one of these factors is deliberately changed, the remaining conditions still need to stay controlled so that its effect can be separated from everything else [35].

The starting state of the system also matters more than it may initially appear. Existing queue backlogs, cached data, stored state or an already-running serverless instance can all change the behaviour of an otherwise identical workload. Tests therefore need to begin from a known state, or use a consistent warm-up process where starting cold would not reflect the behaviour being studied. Repeating each configuration is equally important in cloud environments, where scheduling and background activity can cause some variation even when nothing in the experiment itself has changed. Looking across several runs or full request distributions gives a much stronger basis for comparison than relying on one result [35].

Finally, a distinction should be made between measurements collected directly from the running system and results calculated from those measurements afterwards. Latency, throughput and resource usage can be observed during a test, while measures such as scaling speedup or resource break-even points are derived from that underlying data. Both are useful, but derived results need to make their assumptions clear. Overall, controlled experimentation establishes whether system behaviour changes, while observability provides the information needed to understand why. Together, these

ideas form the basis for both the architecture experiments in this report and the later evaluation of the completed system.

3.7 Research Summary

The literature reviewed in this section provides the foundation for the later design and evaluation plans. Portfolio-management literature shows that investment support should be based on portfolio-level risk, return and diversification rather than isolated asset selection. Investor-profiling and suitability literature shows that recommendations need to account for objectives, constraints, risk tolerance, time horizon, liquidity needs and existing holdings. This supports a decision-support framing where user information and portfolio metrics are compared before a recommendation is considered suitable.

The machine-learning literature shows that financial prediction models require a managed lifecycle. Data must be sourced, cleaned, transformed into features, labelled, validated and monitored. Prediction models can provide useful signals, but financial forecasting is limited by noise, non-stationarity, overfitting and uncertainty. MLOps literature therefore supports experiment tracking, model registries, drift monitoring and retraining as part of the lifecycle.

Agentic AI literature shows how models, tools and calculations can be coordinated through multi-step reasoning. This is relevant when portfolio construction requires several tools and repeated assessment rather than a single prediction. Guardrails and evaluation literature further show that agentic systems should be assessed for task completion, constraint compliance, consistency and explanation quality.

Cloud-deployment literature shows that architecture choices should be evaluated as trade-offs. Monolithic, microservice, event-driven and serverless approaches differ in decomposition, communication and execution model. None is universally better. Performance-testing literature then provides the basis for comparing these architectures through controlled workloads, metrics, observability and reliability testing. Together, these research points provide a clear basis for the thesis design and evaluation plans without making final design decisions inside the literature review.

4. Experimental Evaluation

This section experimentally evaluates the architectural approaches considered for the proposed system. While the previous research identified the characteristics, benefits, and limitations associated with each architecture, these alone are not enough to determine how the approaches behave under the specific workloads expected within this system. The experiments therefore provide a controlled comparison of the main architectural trade-offs under representative user-request and market-data workloads.

The section first defines the common experimental design, including the simplified system, workloads, deployment environment, metrics, and controls used throughout the evaluation. A series of experiments then evaluates the user-request architecture, market-data processing approach, event-driven scalability, workload placement, and Serverless deployment. Rather than selecting the final architecture directly, the results identify the performance, scalability, and resource implications that should be considered when developing the system design in Section 5.

4.1 Experimental Design

This section outlines the setup used for the experiments. It explains the simplified version of the system being tested, how the application workloads and data are represented, and the environment that the tests are being run in. It also sets out the metrics, control variables, assumptions, and limitations used throughout the evaluation.

4.1.1 Objectives and Rationale

The experiments conducted in this project investigate the suitability of various potential cloud computing architectures for the main workloads of the proposed system. The purpose is not to identify a single architecture for the entire system but rather to determine which architectural characteristics are most appropriate for the individual components. The results can then serve as a basis for the system's overall design.

The proposed system contains two primary workload streams. The first stream is a synchronous user-request stream, in which the decision agent accesses supporting tools to generate, evaluate, and rank candidate portfolios that meet the user specifications. The second is an independent market-data stream that processes incoming market updates and ultimately generates and stores asset signals for later access by the agent. Each of these streams has different performance and scalability requirements and will likely therefore benefit from different architectural approaches.

Although the overall system structure has been informed by the literature, experimentation is still required to evaluate the performance, the resource usage, and the scalability trade-offs of the architectural approaches used to implement it. The results provide empirical evidence that can be considered when evaluating the architectural choices made in the subsequent system design.

4.1.2 Experimental System and Omissions

The experimental system is a simplified implementation of the system defined in the thesis specification. It reproduces only the main application elements and communication patterns required to test architectures for the two system components.

Financial models are treated as a constant across architectures and are therefore omitted from the experimental system. Their inference is instead represented using deterministic synthetic workloads that produce measurable CPU and memory demand while remaining identical between experiments.

The decision agent is also simplified by replacing non-deterministic agentic reasoning with a fixed orchestration process that performs the same sequence of signal retrieval, portfolio generation, evaluation, comparison, and ranking operations for each request. This allows the experiments to focus on differences caused by architecture rather than model behaviour or variations in agent reasoning.

Each of the two streams in the system has different requirements and therefore needs to be handled and tested differently, using different base software. User portfolio requests are handled by a simulated deterministic process that represents the decision agent. This process retrieves current asset signals, generates and evaluates candidate portfolios against user preferences, and ranks the results. The same pipeline is used across all architectural tests to keep application behaviour consistent. The market-data stream similarly uses a fixed synthetic inference component rather than dedicated models, making sure that the same processing workload is applied across architectures. For Experiments 1 and 2, the two workload streams are evaluated independently whereas experiment 3 deliberately runs both streams simultaneously to evaluate the effect of compute placement and interference between them. However, all experiments are still calibrated in such a way so that updates to the signal store from the market-data pipeline do not alter the data for the user-request pipeline. This ensures all inputs remain constant between experiments, meaning the only changing factor is the architectural configuration.

4.1.3 Workloads and Data

The following components define the fixed application workload used throughout the evaluation. Different experiments used different groupings of these components.

Component	Experimental role	Represents
User-Request Generator	Produces controlled streams of portfolio requests, at configurable rates.	Incoming user requests
Request Processor	Transforms user-profile information into meaningful user-side signals using a constant process.	User information processing model
Portfolio Generator	Performs fixed computational work to produce candidate portfolios.	Portfolio generation tool
Portfolio Evaluator	Compares signals of the generated portfolios to user-signals and provides scores.	Portfolio suitability Tool
Decision process	Deterministically coordinates signal retrieval, portfolio generation, evaluation and ranking.	Agentic decision agent
Market-Data Generator	Produces controlled streams of market-data, at configurable rates	Incoming live market data

Market-data processor	Takes ingested market stream and performs a fixed synthetic inference workload to produce and then store asset signals.	Asset-level deployed models
Signal Store	Stores most-recent signal values for each stock. Two versions: one for the market-data stream which can be updated and one for the user-request stream which is fixed.	Shared asset-signal storage
Event Broker	Buffers and distributes market updates in relevant experiments	Event-driven messaging infrastructure

Table 4.1: Experimental Components and their Representation

The same datasets and input sequences are reused across matched architectural configurations to ensure comparability. Furthermore, in cases where dataset size or workload complexity is deliberately varied, the same corresponding input is used for each architecture being compared.

4.1.4 Environment and Deployment

The experiments are run using the provided OpenStack and RKE2 Kubernetes environment. OpenStack provides the virtual infrastructure needed for the testing, while Kubernetes is used to deploy and manage the experimental applications. The application components are containerized and deployed using fixed configuration files. This allows each architectural setup to be recreated consistently across the repeated tests.

Comparable experiments are run within the same cluster and network environment. CPU and memory allocations are kept equivalent where the purpose is to compare architectures under the same resource budget. In the case where resource capacity is itself part of the independent variable – that is in the consumer-scaling experiment – the resource allocation of each individual instance is fixed while the number of instances is varied. Load generation is also run separately from the applications being tested, preventing it from using resources allocated to the system itself. Any architecture-specific infrastructure, such as the event broker, is introduced only where required. The exact deployment setup for each architecture is described within the relevant experiment subsection in Section 4.2.

4.1.5 Evaluation Metrics

The experiments use a common set of metrics to compare the performance and resource requirements of each architectural approach. These mainly include latency, throughput, CPU usage, memory usage, and error rate. Latency is measured using percentile values such as p50 and p95, allowing both normal and slow requests to be considered as opposed to relying solely on the average.

Some metrics are relevant only to specific experiments and are therefore introduced only where required. For example, market-data processing also considers queue depth and scaling speedup, while serverless testing considers cold-start latency and has an emphasis on resource usage. The exact metrics used for each test and their relevance are defined in the corresponding experiment subsection in Section 4.2.

4.1.6 Experimental Variables

Experimental variables describe what is being changed within experiments (independent variables), what is being measured (dependent variables), and what must remain constant across experiments (control variables). The independent variable for each experiment is the architectural characteristic being tested, while the dependent variables are the performance and resource metrics being measured. The variables that must be controlled throughout the experiments are:

- The datasets used and the order in which inputs are processed.
- The computational workload performed by each application component.
- Configuration of the user-request and market-data generators.
- Available CPU and memory resources.
- The Kubernetes cluster and surrounding network environment.
- Application, dependency, and configuration versions.
- Test length, including any warm-up before measurements begin.
- The tools used to collect performance and resource metrics.
- The number of times each test is repeated.

These controls are kept consistent across comparable tests so that differences in results can be credited primarily to the architectural characteristic under evaluation.

4.1.7 Assumptions

The experiments make several assumptions to enable the system's architectural behaviour to be tested independently of the final application logic. The synthetic inference workload is assumed to provide a reasonable representation of the computing demand of deployed asset-level models. The deterministic decision process is assumed to represent the final agent's communication and orchestration behaviour. The synthetic user requests and market data are also assumed to provide suitable workloads for testing the system, even though they are not real production traffic. As stated, the components replaced with synthetic inference or synthetic data are not themselves affected by the architectural choices being evaluated. Instead, they provide the workload that each architecture must handle, meaning that controlled synthetic representations can be used without changing the purpose of the comparison.

The same logical workload is kept consistent across comparable architectural configurations, so differences in the results are assumed to be caused by the architecture rather than changes to the application. The shared signal store is also either kept fixed or is reset at the start of each experiment as it is not evaluated as part of the architectural comparison. The OpenStack and Kubernetes environment is assumed to remain stable enough across repeated tests for the results to be compared reliably.

4.1.8 Limitations

The main limitation of the experimental system is the use of synthetic inference and deterministic agent behaviour rather than real financial models and an LLM-based decision agent. This improves repeatability but means the measured latency, CPU usage, memory usage, and scaling behaviour cannot be treated as exact predictions of the final system. Instead, these metrics should be used to compare the relative performance of the architectures rather than as a reference for how the final system will perform. Real models may introduce additional effects such as model loading, higher memory requirements, multithreading, or varying inference times, while a real agent may make a different number of tool calls between requests. The experiments are also limited to the provided OpenStack and Kubernetes environment, meaning the absolute results may differ when deployed on different infrastructure.

The experiments are also limited by the resolution of the infrastructure monitoring data and by the specific Kubernetes deployment environment used. Where short execution phases could not be measured reliably in isolation, resource analysis was performed over longer measured workload cycles. The Serverless resource break-even analysis is therefore a derived sensitivity analysis based on these measured cycles rather than a direct test of every possible idle interval. Similarly, workload-placement results reflect the behaviour of the available Kubernetes topology and should not be assumed to generalise directly to other clusters.

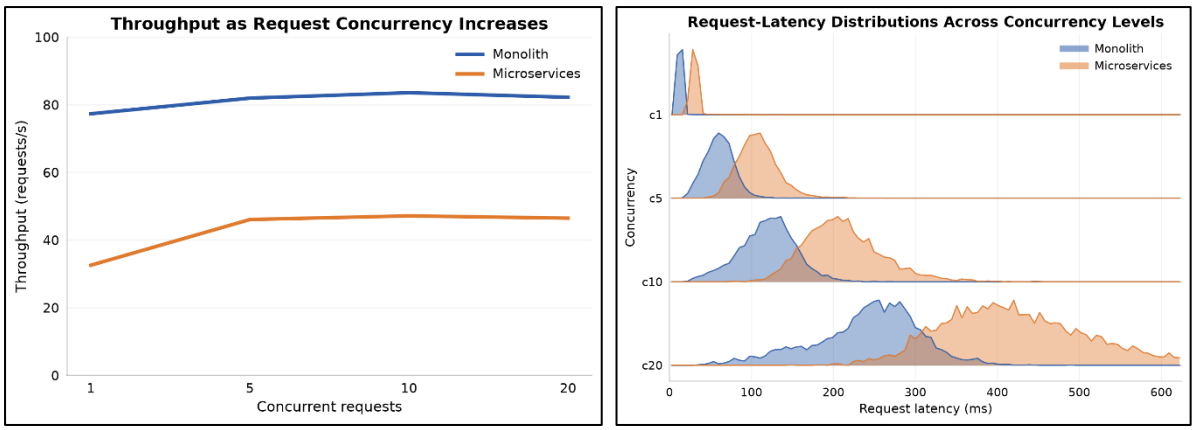
4.2 Architecture Experiments

The following experiments evaluate the main architectural characteristics considered for the proposed system. Each experiment isolates a specific design question and measures its effect on performance, scalability, or resource usage under the controlled setup defined in Section 4.1. Together, the experiments cover the user-request pipeline, market-data processing and scaling, workload placement, and serverless deployment.

4.2.1 Experiment 1 - User-Request Architecture

This experiment evaluates the user-request pipeline under two differing approaches: a single monolithic service versus a distributed microservices architecture. It measures this cost through latency, throughput, and resource demand across varying concurrency levels. While the pipeline's underlying logic naturally divides into distinct microservices, the final decision engine remains hyper-sensitive to latency. Therefore, it is imperative to know exactly how much performance is sacrificed by drawing these service boundaries.

To test this, a deterministic user-request workflow was created which simulated a typical workload that the system would have to handle in production. Both architectures were run using this same workflow. The monolith ran everything – request processing, portfolio generation, evaluation, and coordination – in a single container, while the microservices setup broke these same operations into separate entities tied together by synchronous HTTP requests. Both implementations used the same fixed asset-signal and user-request data, as well as the same application logic. Hardware limits were also imposed, ensuring both setups had identical CPU and memory pools. This consistency made sure that the only changing variable was the architectural deployment method. Once this setup was complete, formal tests were run using the same sequence of 1,000 pre-generated user requests at concurrency levels of 1, 5, 10, and 20. Throughput, request latency, CPU usage, memory usage, and error rate were recorded for each configuration and the aggregates of these results are provided in Figure 4.1 and Table 4.2.



b) Throughput *a) Latency*

Figure 4.1: User-request performance across concurrency levels

Architecture	Concurrent requests	Throughput (requests/s)	p95 latency (ms)	Mean CPU (cores)	Mean memory (MiB)	Throughput per CPU core
Monolith	1	77.36	15.7	0.426	69.9	181.6
Monolith	5	81.98	87.9	0.464	71.4	176.7
Monolith	10	83.59	169.0	0.403	72.0	207.5
Monolith	20	82.23	331.5	0.898	71.9	91.6
Microservices	1	32.59	35.1	0.846	174.7	38.5
Microservices	5	46.10	151.2	1.407	180.1	32.8
Microservices	10	47.17	295.1	1.402	182.4	33.6
Microservices	20	46.49	610.8	1.262	185.1	36.8

Table 4.2: User-request performance and resource usage summary

Across every tested concurrency level, as shown in Figure 4.1a, the monolithic implementation outperformed the microservices setup. The throughput for the monolith approach remained relatively stable as concurrency levels increased. The throughput for the microservices setup, by contrast, had a much larger initial spike before quickly reaching a plateau. Latency increased for both architectures as more requests were processed concurrently, although the increase was noticeably greater for microservices relatively. This is also visible in the latency distributions, where the separation between the two architectures becomes more pronounced at higher concurrency as shown in Figure 4.1b. Resource usage followed a similar pattern, with microservices requiring substantially more memory and CPU despite achieving lower throughput as expressed in Table 4.2.

This type of behaviour was expected however, as separating the synchronous request path introduces additional communication between services, serialization, and multiple application runtimes [22], [23]. Even so, the size of the performance difference and the early throughput plateau were more significant than expected. No stage-level tracing was collected for this experiment, so the exact source of this bottleneck cannot be identified from the results alone.

Ultimately, separating the user-request path into distinct services introduces a measurable performance and resource cost when the individual stages remain synchronously dependent on one another. This is particularly relevant to the proposed decision process, where several tool calls may need to complete before a response can be returned to the user. This does not however mean that the final system should use a monolithic approach, as using microservices provides a lot of other benefits that a monolith implementation does not – as discussed in Section 3.5.2. Instead, this

experiment should be used in a quantitative analysis to assess whether these benefits would be worth it considering the clear latency, throughput, and resource overhead.

4.2.2 Experiment 2A - Market-Data Architecture

This experiment evaluates whether an event-driven market-data pipeline would provide a performance advantage over direct synchronous processing in varying conditions. These conditions represent different potential asset counts and per-asset processing workloads to test which approach works best in which scenarios. The comparison between these two architectures is necessary because the market-data stream operates independently of user requests and has different requirements for data handling – specifically in that it needs to handle large periodic batches of data. And while an event-driven architecture promises better buffering and throughput under load, it also forces the system to take on extra messaging infrastructure and processing weight – something that should be considered in the final design [27], [28].

Two implementations of the market-data pipeline are compared. The Direct approach sends generated market updates directly to the processor, while the Event-driven alternative publishes the same updates through NATS JetStream before they are consumed and then processed. Cycles with dataset sizes of 100, 500, and 1,000 assets are injected into applications with Light, Medium, and Heavy synthetic inference workloads to simulate nine different workload conditions for each architecture. The Event-driven configuration uses one singular consumer so that the experiment evaluates the architectural difference rather than horizontal scaling, which is tested separately in Experiment 2B (Section 4.2.3). As with experiment 1, all other conditions such as application logic, market data, and resource limits, are kept constant across the two architectures.

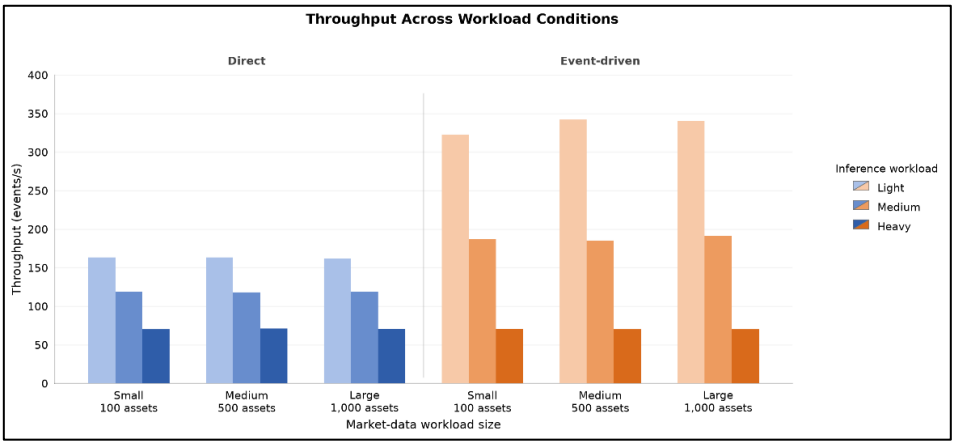


Figure 4.2: Market-data throughput given workload conditions

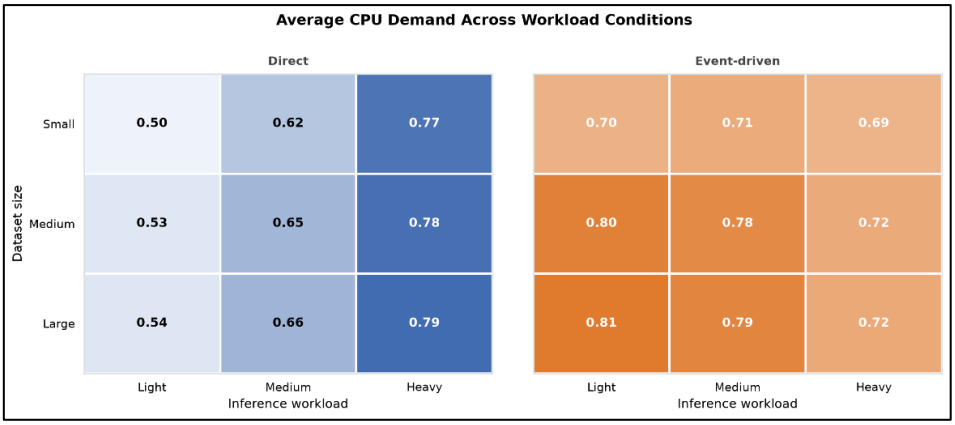


Figure 4.3: Market-data CPU usage given workload conditions

Figure 4.2 shows that the event-driven approach had a much higher throughput than the direct approach at lower inference workload levels, but this performance quickly converged as the per-asset workload grew. Interestingly, the dataset size had almost no effect on the performance for either architecture, indicating a linear relationship between overall processing time and asset count. This suggests that architecture should be decided more heavily based on the inference complexity rather than the asset count. Unsurprisingly, as shown in Figure 4.3, the event-driven model had a higher CPU usage during lighter loads – a logical side effect of running extra messaging infrastructure alongside a faster processing pace. As inference grew heavier, the amount of time spent consuming the market data decreased relative to the time spent processing this data. This reduced the average CPU usage of the event-driven architecture and caused the CPU gap between the two profiles to shrink. Overall, the results suggest that the architectural advantage of Event-driven processing decreases as computation becomes the dominant cost. This behaviour was expected, although the extent to which the two architectures converged under the Heavy workload was notable.

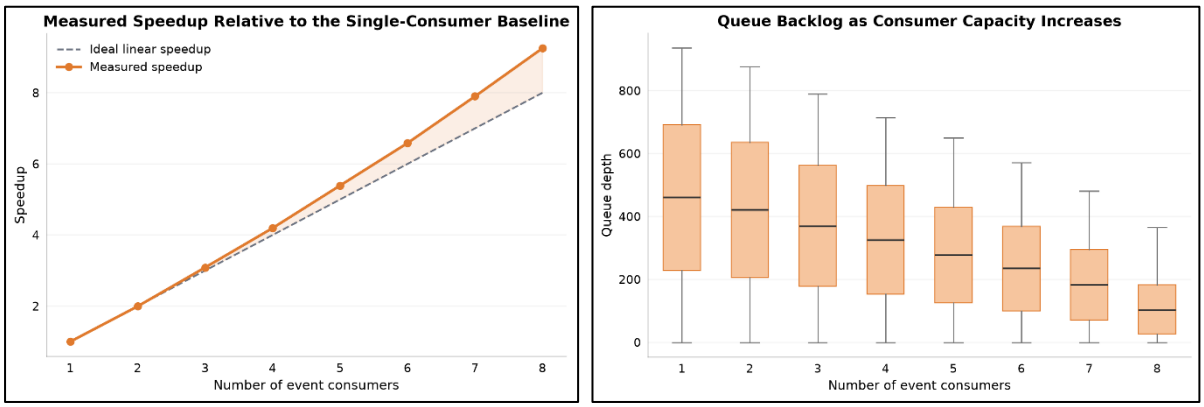
These metrics prove that an event-driven pipeline's usefulness hinges entirely on the computational weight of individual updates – something that will be determined by the complexity of the model in the actual system. If the model is lightweight, the event-driven approach is much more desirable with regards to processing speeds, despite the higher CPU tax. But as soon as the model's workload begins to increase the compute stage itself starts to turn into the primary bottleneck meaning the architectural layout becomes unimportant.

4.2.3 Experiment 2B - Event-Driven Horizontal Scaling

One of the primary benefits of using an event-driven architecture is the ability to use multiple event consumers to handle incoming market data [27]. This experiment evaluates how effectively the Event-driven market-data pipeline scales as these additional consumers are introduced. It specifically looks at whether the additional processing capacity actually clears out queue backlogs and increases throughput. This would indicate whether or not and event-driven approach could handle large influxes of market updates. While Experiment 2A gave us a single-consumer baseline, this test proves whether the architecture can successfully exploit horizontal scaling under pressure.

To keep the processing demand consistently high, the pipeline is tested using the largest dataset of 1,000 assets paired with the heavy synthetic inference workload. The number of consumers is then scaled up from one to eight, with each new consumer receiving an identical, isolated slice of CPU and memory. NATS JetStream and the remaining application configuration were kept unchanged. This isolates consumer count as the primary independent variable – meaning total compute capacity scales

up linearly with every consumer added. The metrics tracked for each experiment were throughput, baseline speedup, queue depth, and CPU demand.



a) Processing Speedup *b) Queue Depth*

Figure 4.4: Event-driven horizontal scaling performance

Throughput scaled very strongly as consumers were added as shown in Figure 4.4a. For the first four consumers, the speedup curve tracked ideal linear scaling almost perfectly before creeping slightly above it as more further consumers were added. At eight consumers, speedup reached roughly 9.3 times the single-instance baseline – safely outperforming the theoretical ideal of 8.0. Naturally, queue behaviour also improved, with Figure 4.4b showing that both the typical backlog and upper queue depth progressively decreased as consumer count increased. The slightly super-linear scaling was surprising but should not be interpreted as evidence of a specific optimisation within the architecture. It may result from fixed overheads being amortised across a larger amount of parallel processing or from better utilisation of the available compute resources. The important observation, however, is that increased consumer capacity consistently yielded higher throughput and reduced backlog under the tested workload.

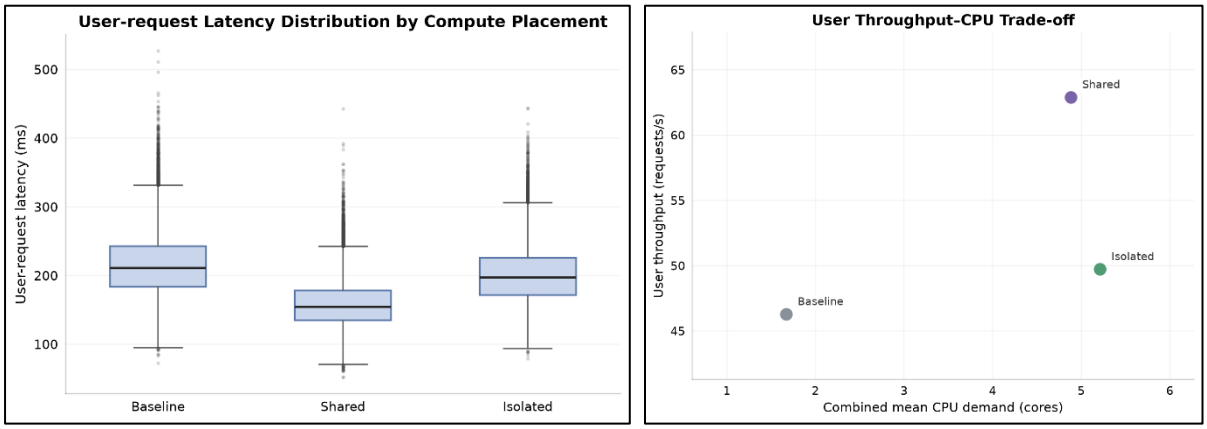
These tests prove that the event-driven pipeline actually scales processing capacity when fed more consumers, rather than just moving the performance bottleneck further down into the message queue. This is relevant for market data, as processing demands shift sporadically depending on asset counts, update frequencies, or changing model complexities. Replicating consumers gives a reliable scaling lever, though it obviously inversely spikes the aggregate compute footprint. When finalizing the system design, this clear performance dividend must be weighed against the cost of running extra infrastructure.

4.2.4 Experiment 3 - Workload Isolation

This experiment evaluates how running the latency-sensitive user-request workload alongside background market-data processing changes user-facing performance, and whether separating the two workloads onto isolated compute resources provides a measurable advantage over allowing them to share the same compute pool. This comparison is necessary because the two streams are evaluated independently in Experiments 1 and 2 but will operate simultaneously in the proposed system. Even if each architecture performs well independently, resource contention between them could affect the responsiveness of the user-facing decision process.

A representative microservices user-request configuration and event-driven market-data configuration are run simultaneously. User requests operate at concurrency 10, while the market-data pipeline uses the medium dataset, with the heavy inference workload, and four event consumers. Three conditions

are compared: a user-only baseline, a shared configuration in which the user and market workloads are placed on the same compute pool, and an isolated configuration in which the workloads are placed on separate worker resources. Application logic and individual resource configurations remain unchanged between shared and isolated conditions so that the primary difference is compute placement. Ten formal repetitions are performed for each condition.



a) Latency *b) Throughput / CPU usage*

Figure 4.5: User-request performance and cost across compute placements

Figures 4.5a and 4.5b show that the result did not follow the expected ordering. The shared configuration produced the highest user throughput at approximately 62.9 requests/s, compared with 49.7 requests/s for Isolated and 46.3 requests/s for the user-only Baseline, despite requiring substantially more combined CPU than the baseline. User-request latency followed the same pattern, with the shared configuration showing a clearly lower latency distribution than both the baseline and isolated conditions. This was unexpected because isolation was intended to reduce interference between background processing and user requests. The result may reflect Kubernetes scheduling behaviour, locality, differences in resource utilisation, or other characteristics of the specific cluster placement. However, the collected metrics cannot identify one of these mechanisms as being the cause. The result should therefore be treated as a measured property of the tested deployment rather than a general conclusion that shared compute improves user-request performance.

The experiment produces no evidence that compute isolation improved the user-facing workload under the tested configuration. Instead, the shared placement produced the strongest user performance, while the isolated configuration provided only a smaller improvement over the baseline. This shows that separating workloads across compute resources does not automatically improve latency-sensitive performance and that placement behaviour can depend strongly on the surrounding Kubernetes environment. For the proposed system, the result identifies workload placement as a configuration that should be considered alongside the actual deployment environment rather than treated as an architectural benefit in isolation.

4.2.5 Experiment 4 - Serverless Suitability

This experiment evaluates the performance and resource trade-off introduced by deploying a representative stateless system tool using a serverless scale-to-zero approach rather than keeping the service continuously available. It tests the differences in cold-start cost, warm-request behaviour, idle resource saving, and the amount of idle time required before the released resources offset the activation overhead. This comparison is necessary because some tools within the proposed system may only be accessed intermittently. Serverless deployment could therefore reduce resources

consumed while these tools are unused, but only if those savings are sufficient to justify the additional cost of reactivating the service when a request arrives [30], [31].

The Portfolio Evaluator is used as the representative stateless service and is deployed using the same application image and resource configuration in both treatments. The always-running configuration uses a standard Kubernetes deployment with one continuously available replica, while the serverless configuration uses Knative with scale-to-zero enabled and a maximum of one active replica. Each formal run contains three request bursts, with 100 warm requests at concurrency 10 following the first request after an idle period. Ten repetitions are performed for each deployment model. Request latency and throughput are measured directly, while cAdvisor resource measurements are used to derive cycle-level CPU-seconds and MiB-seconds. The resource sensitivity analysis then estimates how the total resource cost changes as the no-request interval between bursts increases.

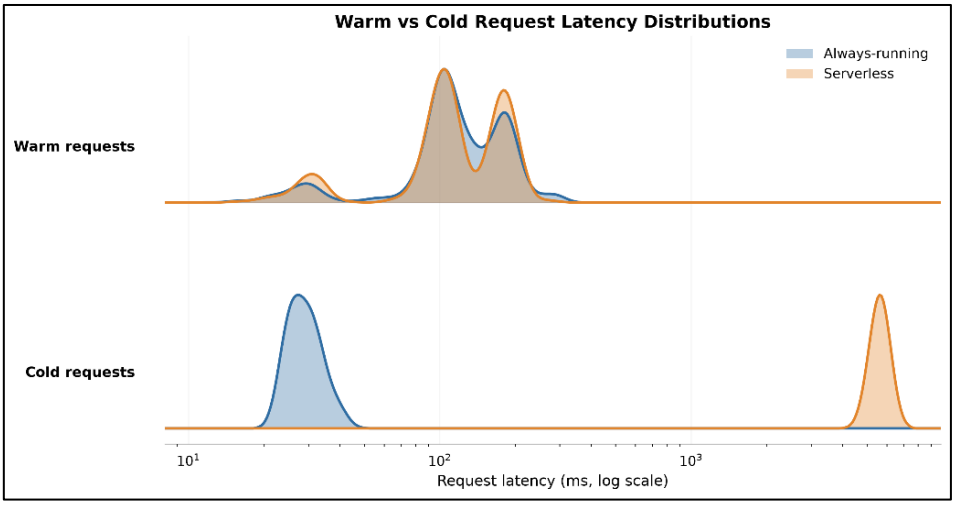


Figure 4.6: Warm and cold request latency by deployment model

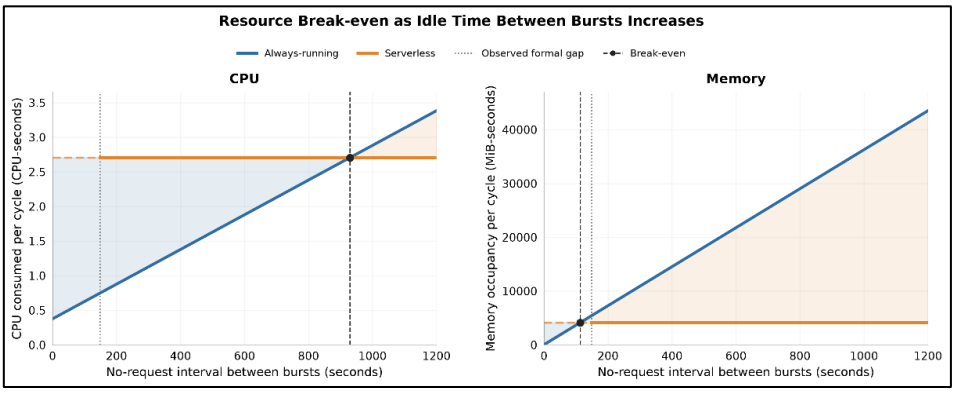


Figure 4.7: Resource break-even levels given deployment model

Figure 4.6 shows the request latency for warm and cold requests under each deployment model. Warm performance was almost unchanged between the two deployment models, with the latency distributions closely overlapping once the Serverless instance was active. Cold requests were substantially different. The Always-running service returned these requests in tens of milliseconds, while Serverless requests clustered at approximately 5–6 seconds because the service first had to scale from zero. This cold-start penalty was expected, while the comparable warm behaviour confirms that the difference is primarily associated with deployment and activation rather than application processing. The resource analysis shown in Figure 4.7 expresses a different trade-off for CPU and memory. Always-running continues accumulating resource cost while idle, whereas the serverless

serving footprint stops increasing after scale-to-zero. The estimated CPU break-even occurs at approximately 929 seconds of no requests, while memory reaches break-even much earlier at approximately 113 seconds. The long CPU break-even occurs because the always-running evaluator uses very little CPU while idle, so a long zero-replica period is required to recover the CPU spent during serverless activation. Memory remains allocated while the always-running service is resident and therefore reaches break-even much sooner.

The experiment shows that serverless suitability cannot be assessed using idle resource reduction alone. Scale-to-zero removes the serving footprint during sufficiently long inactive periods but introduces a substantial latency and CPU cost when the service must be reactivated. The resulting trade-off also differs by resource: memory savings become favourable at considerably shorter idle intervals than CPU savings. For the proposed system, this means the relevance of Serverless deployment depends on the access frequency and latency requirements of the individual tool being considered. The results provide a basis for evaluating those characteristics in the later design without establishing that all stateless components should either use or avoid Serverless deployment.

4.3 Overall Findings and Design Implications

Overall, the experiments demonstrate that the architectural approaches introduce different trade-offs depending on the type of workload being handled. For the synchronous user-request path, introducing service boundaries resulted in a clear latency, throughput, and resource overhead, with this difference becoming more significant as concurrency increased. The market-data experiments showed a different pattern. Event-driven processing provided a considerable throughput advantage while inference remained relatively lightweight, but this advantage reduced as computation became the dominant part of the workload. At the same time, the event-driven pipeline responded strongly to horizontal scaling, with additional consumers consistently increasing processing capacity while reducing queue backlog. Together, these results show that the same architectural characteristics do not provide equal value across the two primary system workloads.

The later experiments also highlight the importance of deployment behaviour beyond the basic application architecture. Compute isolation did not provide the expected improvement to user-facing performance, with the shared placement instead producing the strongest result under the tested environment. As the cause could not be isolated, this finding is more useful as evidence that workload placement should be evaluated in the context of the final Kubernetes environment rather than assumed to provide an inherent performance benefit. Serverless deployment presented a similarly conditional trade-off. Warm performance remained comparable once the service was available, but scaling from zero introduced a substantial cold-start cost. Resource savings were also dependent on how long the service remained unused, with memory becoming favourable considerably earlier than CPU.

These findings provide a set of practical considerations rather than a single preferred architecture. Synchronous communication overhead, inference complexity, horizontal scalability, workload placement, access frequency, and idle resource demand all affect the suitability of the different approaches. The system design presented in the Section 5 can therefore consider these experimental results alongside the requirements and architectural characteristics identified through the earlier research. In this way, the final architecture can be developed around the requirements of each system component rather than applying the same architectural model throughout the entire application.

5. Design Plan

This section defines the overall plan for the design of the final system. It combines the research from the literature review in Section 3 with the findings from the experiments in Section 4 to propose a system capable of meeting the functional and non-functional requirements laid out in Section 2. Some design decisions will still need to be validated during development. As such, each area is given a proposed initial design, but relevant alternatives and the testing required to determine the most suitable option are also identified if necessary. The section first defines the overall system architecture before specifying the design of each major component.

5.1 Overall System Design

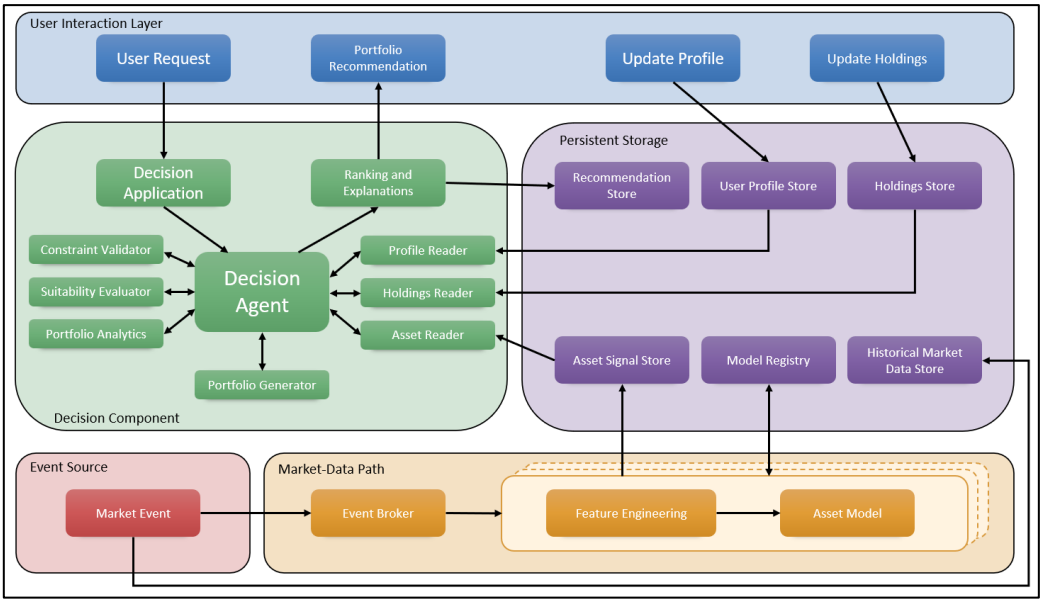


Figure 5.1: Overall Broad System Design

The proposed system will use a hybrid cloud-native architecture, rather than applying one architectural approach to every component. The system can be divided into two main processing paths: a synchronous user-request path and an independent market-data path. These streams have different requirements, with user requests being more sensitive to response time while market-data processing is more dependent on throughput and scalability. The two paths should therefore be designed independently while sharing the information required to produce recommendations.

The user-request path will centre around a single decision application responsible for producing recommendations. When a recommendation is requested, the decision agent will use its supporting tools to retrieve the user's profile, current holdings and latest available asset signals. The initial design uses one agent to coordinate the supporting portfolio-generation, analytics, suitability and validation tools required to construct and assess potential portfolios. Candidate portfolios can then be compared and ranked before the selected recommendations and supporting explanations are returned to the user. These components should remain logically separated within the application even where they are initially deployed together, allowing individual parts to be tested or separated later if required.

Market data will be handled through a separate background processing path so that market updates do not need to be processed while a user waits for a recommendation. Incoming observations will first be collected and converted into a consistent internal format before being passed through an asynchronous event-driven pipeline. Processing workers will then perform the required feature

calculations and model inference before generating updated asset-level signals and storing the resulting information. The processing workers will be horizontally scalable so that additional capacity can be introduced as the workload increases. This structure follows the market-data experiments in Section 4, with the exact messaging, processing and storage technologies expanded on later in this section.

The two processing paths should remain largely independent and communicate through shared system state rather than directly invoking one another. In particular, the market-data path will continuously update asset signals, while the decision application will access the most recent available signals when processing a user request. Persistent storage will also be required for user profiles, portfolio holdings, historical market information and recommendation records. Model training and model management will operate as a supporting process, supplying validated model versions to the market-data components rather than forming part of the user-request workflow itself. Monitoring, logging and tracing will span both paths so that behaviour can be observed across the complete system.

The system should be implemented as a set of containerised components that can be deployed within a common cloud environment while retaining different deployment characteristics where required. Latency-sensitive user-facing components should initially remain continuously available, while background processing capacity should be able to scale independently with workload. More specialised approaches such as compute isolation, additional service decomposition, caching or scale-to-zero should only be introduced where testing of the completed application shows that it adds a meaningful benefit. This follows the findings in Section 4, where deployment choices were shown to depend strongly on the workload and surrounding environment. The following sections therefore define the proposed design of each component in more detail and identify the alternatives that still require validation.

5.2 User Profile and Portfolio Management

The user profile must give a complete picture of all the data needed to judge portfolio suitability. Based on the requirements in Section 2 and the suitability factors identified in Section 3.2, the profile will specifically need to contain the user's investment objectives, risk tolerance, ability to bear losses, investment horizon, liquidity needs, available budget, investment experience and any explicit asset or sector constraints as the minimum standard. Current portfolio holdings should be maintained alongside the profile as a separate input so that recommendations can account for the user's existing investment position rather than considering potential portfolios in isolation.

User-entered information will then be converted into a consistent set of internal profile signals that can be compared with portfolio characteristics. The initial implementation will use deterministic mappings rather than a learned profiling model, producing values such as risk tolerance and capacity, investment horizon, liquidity requirements, concentration limits and investment preferences. This approach provides a clear and repeatable link between the user's answers and the later suitability assessment, which is particularly important for explaining why a recommendation was produced. A learned profiling model could be evaluated as an alternative if the rule-based representation proves unable to capture sufficient variation between users.

Current holdings must be handled separately from the user's stated preferences and will set the starting state of the portfolio. Each holding will record the asset with its quantity or current value. From this, the system can derive portfolio weights using the latest available market information. Deterministic portfolio calculations must then produce the current allocation, concentration, exposure, risk and diversification features of the user’s current portfolio as identified in Section 3.2.4.

These figures provide a baseline that new portfolio options can be compared to, allowing the system to recommend changes relative to the user's existing position rather than generating recommendations without context.

The processed profile, current holdings and derived portfolio state will be stored persistently and retrieved as a single user context when a recommendation is requested. The system will include input validation that will prevent incomplete or invalid profiles from reaching the decision process. Once produced, each of the recommendations will retain the profile and holdings version used to produce it so that the result can later be reproduced and explained. The exact persistence technology is considered with the wider data-management design, but the interface exposed to the decision process should remain independent of that implementation. The selected profile fields and deterministic mappings will later be validated using fixed investor scenarios, as expanded on in Section 6.

5.3 Market-Data Pipeline and Predictive Models

The market-data pipeline will use aggregated OHLCV data rather than tick-level or order-book data. Daily observations are proposed as the default as the system is focused on portfolio-level investment decisions rather than short-term trading. This also keeps the processing requirements lower and avoids adding unnecessary market-data complexity and noise, as discussed in Section 3.3.2. yFinance will initially be used to obtain historical and recent market data, but access will be placed behind a common provider interface so that another public API or higher-quality commercial feed can be introduced without changing the rest of the processing workflow. The initial asset universe will contain a fixed, configurable set of liquid equities with sufficient historical data. A separate predictive model will first be trained for each asset to capture relationships specific to that stock. This will then be compared against a single model trained across all assets to determine whether the additional model-management complexity actually provides a meaningful predictive benefit.

After ingestion, market observations will be validated and converted into a consistent representation before being used for feature calculation. The initial model input will consist of 1-, 5-, and 20-day returns, price relative to its 20-, and 50-day moving averages, 20-day rolling volatility, and current volume relative to its 20-day average. These features provide a relatively small set of price, momentum, risk and volume information without unnecessarily increasing the complexity of the model. All rolling calculations must only use information available at the time of each observation to prevent information leakage. These features and window lengths represent the proposed initial configuration rather than fixed optimal values. They should therefore be externalised through configuration so that their effect can easily be tested without changing the processing implementation.

The predictive component will estimate the percentage return of each asset over the following five trading days. A continuous return prediction is preferred to a direct buy, hold or sell classification because it provides a reusable asset-level signal that can later be considered alongside the user's profile and portfolio characteristics. Quantities that can be derived directly from the available data should remain deterministic. This includes asset-risk which will initially be represented by the twenty-day rolling volatility, while correlation, concentration, diversification and portfolio exposure will be also be calculated deterministically when portfolios are evaluated. This limits the role of machine learning to predicting future information rather than reproducing calculations that can be performed directly. It is also important to note that the five-day horizon will be treated as the initial configuration and compared with one-day and twenty-day horizons during model evaluation to assess how the time horizon relates to accuracy

LightGBM regression is proposed as the primary predictive approach, with a separate model initially trained for each asset, because the input consists of structured, engineered market features and this approach provides nonlinear modelling capability without requiring the complexity of a sequential neural architecture. A separate linear regression model will also be trained for each asset as a simple predictive baseline, while a single LightGBM model trained across the full asset universe will provide a comparison for the per-asset modelling approach. These configurations will be evaluated using walk-forward validation, with the final approach selected based on predictive performance, inference latency and resource requirements. More complex sequential or time-series models considered in Section 3 could be introduced if the initial approaches provide insufficient predictive performance, but they are not included in the initial implementation. This is particularly relevant to the findings from Section 4.2.2, where increasing inference complexity reduced the performance advantage of the event-driven architecture.

During operation, each new market observation will be published to the asynchronous processing pipeline after ingestion and validation. One or more processing workers will update the required features, execute the approved predictive model, calculate the deterministic risk measures and persist the resulting asset signal. NATS JetStream is proposed as the event broker because it was already validated within the experimental environment and supported the horizontally scalable consumer design evaluated in Section 4.2.3. Kafka provides an alternative if substantially greater streaming scale or functionality is required, but this additional complexity is not justified by the current system requirements. Each stored asset signal will contain the asset identifier, latest price, predicted five-day return, risk represented by the twenty-day rolling volatility, update timestamp and model version.

Model training will remain separate from runtime market processing. Training runs will use chronological walk-forward validation, after which the approved model and its associated metadata will be versioned before being made available to the processing workers. The version used should be recorded with every generated asset signal so that recommendations can later be traced to the predictive information available at that time. Historical market observations and generated signals should be retained for reproducibility, model evaluation and drift monitoring. A separate cache of current signals should only be added if later user-request testing shows that access to the persistent store creates a meaningful latency bottleneck. The exact persistence and model-management technologies are defined in Section 5.6.

5.4 Agentic Decision Process and Portfolio Tools

The initial decision process will use a single tool-using agent responsible for coordinating portfolio construction and assessment. When a recommendation is requested, the agent will use its reader tools to retrieve the processed user profile, current holdings and most recent asset signals. Rather than performing financial calculations directly, the agent will call a fixed set of deterministic tools to generate, analyse and validate candidate portfolios before producing the final recommendation. This separates the flexible reasoning process from calculations that need to remain reproducible and consistent. The underlying language model will be accessed through a common model interface so that the decision workflow does not depend on a particular provider.

Tool	Purpose
Profile Reader	Retrieves the user’s preferences and constraints and processes them
Holdings Reader	Retrieves the user’s current holdings and derived portfolio state
Asset Signal Reader	Retrieves the latest available asset signals

Portfolio Generator	Produces feasible candidate portfolio allocations
Portfolio Analytics	Calculates portfolio return, risk, diversification and exposure
Suitability Evaluator	Scores alignment between a portfolio and the user profile
Constraint Validator	Rejects portfolios that violate any hard constraints

Table 5.1: Decision-Agent tools

The agent will initially have access to the seven defined tools listed in Table 5.1. The three reader tools provide all the context to the agent without allowing it to access the underlying stores directly. The remaining four tools handle portfolio construction and assessment with regard to suitability and constraints. All numerical calculations return structured outputs that the agent can compare. This design restricts the agent to a defined tool interface, which also makes the individual decision stages easier to test. It also prevents the agent from making its own unsupported calculations.

Candidate portfolios will be generated using the dedicated Portfolio Generator rather than allowing the agent to assign asset weights itself. The generator will combine the predicted returns from Section 5.3 with portfolio risk calculated from historical returns to create several possible allocations. Portfolio weights must sum to 100%, as short selling will initially be excluded. Explicit asset or sector restrictions will also be enforced and checked by the Constraint Validator, and maximum concentration limits will be taken from the processed user profile. Assets with missing or stale signals will also be excluded from generation. The generator will return multiple suitable portfolio options that represent different return-risk trade-offs, rather than producing a single supposedly optimal portfolio. This gives the decision process alternatives to compare while also keeping the actual portfolio construction reproducible.

Each candidate will then be assessed by separate analytics, suitability and validation tools. The Portfolio Analytics tool will calculate the candidate's predicted return, volatility, allocation, concentration, diversification and relevant exposure measures using deterministic calculations. The Suitability Evaluator will compare these characteristics with the processed user profile and return a structured suitability score, allowing candidates to be compared. The initial score will combine risk alignment, return-objective alignment, investment-horizon and liquidity alignment, diversification and concentration preference using configurable weights. Hard requirements will be handled separately by the Constraint Validator. Any portfolio that violates an explicit budget, exclusion, concentration or other hard user constraint will be rejected regardless of the agent's assessment, preventing the reasoning model from overriding requirements that should be enforced deterministically.

The decision process will operate as a bounded generate-evaluate-refine loop based on the ReAct framework. After retrieving the required user and market context, the agent will request an initial set of portfolios and pass each candidate through analytics, suitability assessment and hard validation. Invalid candidates will be removed, and if no feasible portfolios remain after doing this, then the information will be returned to the generator. The initial implementation will use a maximum of three generation and evaluation cycles to prevent potential uncontrolled reasoning loops. Valid candidates will then be ranked primarily using their structured suitability assessments, with their predicted return and portfolio characteristics retained for comparison. Importantly, the agent will use these results to select the recommendations presented to the user rather than replacing the deterministic evaluation with its own numerical judgement.

The final explanation will be generated from the structured outputs returned by the tools, including the user's objectives, changes from the existing portfolio, predicted return, risk, diversification and any

constraints that affected the result. The information used during each decision should also be retained so that a recommendation can be traced to its profile version, holdings snapshot, asset signals, predictive model and portfolio assessments. The main architectural alternative is a two-agent workflow in which a second agent independently assesses the portfolios proposed by the first and returns feedback for refinement. This may provide an additional reasoning layer, but it also introduces further model calls, latency and complexity. The single-agent design with an independent deterministic validator will therefore be used initially, with the two-agent approach only adopted if evaluation shows a meaningful improvement in suitability, constraint compliance or consistency.

5.5 Recommendation Output and User Interface

The system will return a ranked set of portfolio recommendations rather than individual trade instructions. Each recommendation will contain the proposed asset allocation, the main changes from the user's current portfolio (if provided), predicted five-day return alongside historical portfolio return, portfolio risk, diversification and concentration measures, suitability score and the result of the constraint checks. Where appropriate, these results will primarily be presented through visualisations to make comparisons between the current portfolio and recommended alternatives easier to understand. A supporting explanation will summarise why the portfolio is considered suitable and identify the main factors that influenced its ranking. These explanations will be grounded in the structured outputs produced by the decision tools rather than relying on unsupported free-form reasoning from the agent.

The prototype user interface will be implemented using Streamlit. This provides a lightweight Python-based interface that can integrate directly with the system while avoiding unnecessary frontend development for functionality that is not central to the thesis. The interface will allow users to create or update their profile, enter current holdings, request portfolio recommendations and compare and review the resulting portfolio metrics and explanations. The interface will also provide the deployed model version and its main validation metrics. A dedicated web frontend could replace Streamlit in a production system, but the additional implementation complexity is not required for the planned evaluation.

Each recommendation will also be assigned a unique identifier and timestamp, and will be linked to the profile, holdings, asset signals and model versions used to produce it. The user-facing view should present the information required to understand and compare recommendations without exposing unnecessary implementation detail. However, the underlying technical metadata will remain available for reproducibility and system evaluation. No functionality for executing trades or automatically modifying external investment accounts will be included. The user instead will have to execute the trades themselves if they choose to, and then manually update their holdings.

5.6 Data and Model Management, and System Integration

PostgreSQL will store the user profiles, holdings, historical market data, asset signals and recommendations produced by the system. These will be kept in separate tables, with versions retained where needed so that earlier recommendations can still be linked to the data available when they were generated. A separate in-memory cache will not be added initially, as the extra synchronisation and deployment complexity is not justified at this stage. Redis can be introduced later to hold the latest asset signals if testing shows that reading them directly from PostgreSQL is affecting user request latency.

MLflow will be used to track the predictive-model lifecycle and manage approved model versions. Each training run will record the feature configuration, prediction horizon, model parameters, validation results and resulting model artefact. Models that meet the required evaluation criteria can then be registered and versioned, with the approved model made available to the market-processing workers. The model version used will also be recorded with each generated asset signal, allowing later recommendations to be traced back to the model that contributed to them. Model artefacts and MLflow metadata will therefore be stored persistently so that previous versions can be reproduced when required. Deployed models will also be monitored for changes in their input and prediction distributions, as well as prediction error once the true outcome is available. Sustained degradation will be used to identify when retraining may be required, with the monitoring and retraining process expanded on in Section 6. Any replacement model must still pass the same validation and registration process before deployment.

FastAPI will provide the interface between the Streamlit frontend and the decision application, using synchronous HTTP requests for user-facing operations. The individual decision tools will remain internal modules of the application rather than separate network services, avoiding the communication overhead identified in Section 4.2.1. The market-data path will continue to communicate asynchronously through NATS JetStream and will share information with the decision application through persisted asset signals rather than direct service calls. Common typed schemas will be defined for profiles, holdings, market events, asset signals, candidate portfolios, evaluations and recommendations so that data structures remain consistent across the system.

System behaviour should be controlled through external configuration rather than hard-coded values. This will include the market-data frequency, feature windows, prediction horizon, approved model selection and versions, portfolio-generation limits, suitability parameters and agent iteration settings. Credentials and API keys must be stored separately from normal application configuration. Each recommendation will also retain the relevant configuration, profile version, holdings snapshot, asset-signal timestamps and model version required to reproduce the conditions under which it was generated. The deployment-specific handling of configuration and secrets is defined in Section 5.7.

5.7 Deployment, Scaling, and Observability

The completed system will be containerised and deployed on the provided RKE2 Kubernetes cluster, managed through Rancher and hosted on OpenStack. The user-request application and market-processing workers will be deployed separately so that each can be scaled and configured independently. Supporting services for storage, messaging and model management will also run as separate components where needed. CPU and memory requests and limits will be set for each workload so that resource use remains controlled during deployment and testing. ConfigMaps will store non-sensitive configuration, while credentials and API keys will be kept in Kubernetes Secrets.

Kubernetes health checks will be used to detect failed application instances and restart them where necessary. Market-data events will only be acknowledged once processing and storage have completed successfully, allowing failed updates to be retried. Stateful services will use persistent storage so that important data is retained if a pod is replaced. Application failures will also be recorded through the logging and monitoring setup so that they can be identified during operation and testing.

Scaling will initially focus on the market-processing workers, as the results in Section 4.2.3 showed that adding consumers can increase processing capacity and reduce queue backlog. Fixed replica counts will first be used to measure the behaviour of the completed system, after which Horizontal Pod Autoscaling can be tested using processor utilisation and queue backlog. The user-request application

can also be replicated if concurrent-request testing shows that additional capacity is needed. Both workloads will initially run on the same worker resources with defined CPU and memory limits, as Section 4.2.4 did not show a clear benefit from separating them. Dedicated nodes or placement rules will only be introduced if later testing shows that resource contention is affecting performance.

Scale-to-zero will not be used on the main user-request path because the cold-start delays observed in Section 4.2.5 would increase recommendation latency. Components required to process interactive requests will therefore remain running. Knative may still be used for infrequent stateless background tasks where long idle periods make scale-to-zero worthwhile, but it will not form part of the initial deployment.

OpenTelemetry, Prometheus and Grafana will be used to monitor the deployed system. The user-request path will record overall and stage-level latency, throughput and failures, while the market-data path will also record update latency, inference time, queue depth and signal staleness. CPU, memory, network and storage use, together with active replica counts, will be collected for both workloads. Structured logs and trace identifiers will make it possible to follow individual requests and market updates through their processing stages. This is important because the experiments in Section 4 showed that overall measurements alone were not always enough to identify where performance overhead was being introduced.

5.8 Design Overview

Area	Proposed Design
Overall Architecture	Hybrid design with separate user-request and market-data paths
User Context	Deterministic profile processing with persistent holdings
Market Data	Daily OHLCV from a configurable set of equities
Predictive Models	Per-asset LightGBM, compared with linear and shared-model baselines
Market Processing	NATS JetStream with horizontally scalable processing workers
Decision Process	Single tool-using agent with deterministic generation and validation
Portfolio Assessment	Return, risk, diversification, exposure, suitability and hard constraints
User Interface	Streamlit frontend with FastAPI backend
Data & Models	PostgreSQL for application data and MLflow for model management
Deployment	Containerised on RKE2 Kubernetes/Rancher over OpenStack
Scaling & Placement	Shared workers initially; scale independently where required
Observability	OpenTelemetry, Prometheus, Grafana and structured logging

Table 5.2: Summary of the Proposed System Design

The resulting design provides a complete starting architecture for implementing the final system while retaining flexibility in the areas that still require validation. The main design choices favour simple and measurable components, with additional architectural complexity introduced only where testing demonstrates a clear benefit. The implementation can therefore proceed using the technologies and structures defined in this section, while the main remaining design decisions are evaluated as part of the thesis. Section 6 defines how the completed system and selected alternatives will be assessed.

6. Evaluation Plan

This section defines how the completed thesis system will be evaluated against the functional and non-functional requirements established in Section 2. Evaluation will be performed at several levels, beginning with verification of individual components and their integration before assessing the predictive models, agentic decision process and complete deployed system. Particular attention will be given to systems performance, where controlled workloads and observability data will be used to determine how the application behaves as demand increases, where limitations occur, and whether the final cloud-native implementation satisfies its operational requirements.

6.1 Evaluation Approach and Methodology

Different parts of the system require different forms of evaluation. Deterministic application logic can be checked against known inputs and outputs, while predictive models must be assessed using unseen chronological data. The agentic decision process must instead be tested through repeatable investor scenarios that show whether recommendations remain suitable, constrained and explainable. Finally, the deployed system must be evaluated under representative workloads to determine its latency, throughput, scalability, reliability and resource usage. These levels will be considered together so that a system component is not treated as successful simply because it operates correctly in isolation.

Evaluation Area	Purpose	Measures
Functional	Confirm the deterministic application aspects work as intended	Unit, integration and end-to-end tests
Predictive Model	Evaluate asset predictions	MAE, RMSE, inference cost
Agentic	Evaluate recommendation quality	Suitability, constraint compliance, and consistency
System	Measure system performance	Latency, throughput, scaling, resource use, reliability

Table 6.1: Evaluation Areas

When running formal comparisons, only the factor being tested will change, making sure all other aspects such as application logic and resource limits are kept constant. Each test will begin from a defined constant state, such as using cleared queues or known persisted data. A consistent warm-up period will also be used when cold execution does not represent normal operation. Load generation will run separately from the tested application so that the resources used to create the workload do not affect the results.

Each formal configuration will be repeated five times to account for variation in the cloud environment. Latency will be treated as a distribution rather than represented only by an average, with p50 and p95 reported for all request-based tests and p99 added where the slower tail is relevant. Throughput, error rate and resource usage will be collected over the same measurement interval so that performance is interpreted in relation to the workload being applied.

The evaluation will also combine end-to-end measurements with observability from inside the application. Application behaviour will be assessed through request rate, latency and errors, while CPU, memory, network, storage and saturation indicators will be monitored at the resource level. OpenTelemetry traces, Prometheus metrics, Grafana dashboards and structured logs will then be used

to relate changes in overall performance to individual processing stages. Following the systems-performance approach discussed in Section 3.6, the intention is not only to identify that performance changes, but to provide enough information to understand where the limiting behaviour is occurring.

6.2 Functional and Integration Verification

Functional testing will first be used to verify the deterministic parts of the application independently. Unit tests will cover profile mappings, market feature calculations, portfolio metrics, portfolio generation, suitability scoring, constraint validation and input or schema validation. For these components, fixed inputs will be paired with known expected outputs so that calculation and validation errors can be detected before the complete workflow is evaluated.

Integration tests will then verify the main boundaries between components. This includes reads and writes to PostgreSQL, publishing and consuming events through NATS JetStream, loading approved predictive models, FastAPI communication and the interfaces between the agent and its supporting tools. These tests will focus on whether data is transferred correctly and whether failures are returned in a controlled way rather than on performance.

User-profile and holdings functionality will also be tested by creating, retrieving and updating both forms of information and confirming that the correct versions are stored and returned. The model-management workflow will be verified by completing a training run, recording its configuration and results in MLflow, registering the resulting model and loading a specified approved version for inference.

Finally, end-to-end scenarios will run the complete workflow from user profile and holdings input through market-signal retrieval, portfolio generation, assessment, ranking and explanation. The interface will also be checked to ensure that the required portfolio analysis, model information and recommendation outputs are presented correctly. Together, these tests will provide direct verification of functional requirements FR-01 to FR-09 before the system is subjected to the model, agentic and performance evaluations below. As stated in Section 2, evaluation of this complete workflow is required rather than only evaluating the isolated component functionality.

6.3 Predictive Model Evaluation

Predictive-model evaluation will determine which modelling configuration should be used by the final market-data pipeline. Historical OHLCV data will be evaluated chronologically using walk-forward validation so that every prediction is produced from information that would have been available at that point in time. The same validation and test periods will be used when comparing models, preventing differences in market conditions from being mixed with differences in model behaviour.

Two main comparisons will be performed. First, per-asset linear regression will provide a simple baseline for the proposed per-asset LightGBM model. This will show whether the additional nonlinear modelling capability of LightGBM actually provides a meaningful reduction in prediction error. The second test will compare the per-asset LightGBM approach to a single LightGBM model trained across the full asset universe. This will determine whether maintaining separate models for individual assets produces enough predictive benefit to justify the additional training, storage and model-management complexity. The proposed five-day prediction horizon will also be compared with one-day and twenty-day horizons using the same validation procedure.

Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) will be the primary prediction metrics because the model produces a continuous future-return estimate. Directional accuracy may

also be reported as a secondary measure to show how often the predicted sign of the return matches the observed outcome. Predictive quality will not be considered in isolation. Inference latency and resource usage will also be measured because Experiment 2A showed that increasing inference cost can reduce the performance advantage of the event-driven market-processing architecture. The final configuration will therefore be selected based on whether improved predictive performance is large enough to justify its operational cost.

After deployment, the feature distributions, prediction distributions and realised prediction error will also be monitored. These results will be compared with the behaviour recorded during model validation to identify sustained changes in the data or predictive performance. Where this shows that the deployed model is no longer performing consistently with its validated behaviour, retraining will be considered. Any replacement model must still pass the same walk-forward evaluation and registration process before deployment. This maintains the model lifecycle established in Section 3.3, where monitoring and retraining remain part of model evaluation after deployment.

6.4 Agentic Decision and Recommendation Evaluation

The agentic decision process will be tested using a fixed set of investor scenarios rather than judging individual recommendations subjectively. S0 will provide the baseline, representing an investor with moderate risk tolerance and capacity, a long-term growth objective, a long investment horizon, low liquidity needs, moderate concentration preferences, diversified existing holdings and no explicit exclusions. The remaining scenarios will each change one aspect of this profile so that the effect of that change can be assessed while the market state and other profile information remain the same.

Scenario	Testing Factor
S0	Baseline moderate investor
S1	Lower risk tolerance
S2	Lower ability to bear losses
S3	Higher return objective
S4	Shorter investment horizon
S5	Higher liquidity requirement
S6	Lower concentration tolerance
S7	Concentrated existing holdings
S8	Explicit asset or sector exclusion

Table 6.2: Testing Scenarios

Each scenario will be run ten times using the same market state. Recommendations must first satisfy all hard constraints, including budget, exclusions and concentration limits. Any violation will be treated as a failed recommendation, with full compliance expected because these checks are handled outside the language model. The remaining recommendations will then be assessed using their suitability scores and how they change relative to the baseline scenario. For example, lowering risk tolerance should generally produce lower-risk portfolios when the other inputs remain unchanged. Similar comparisons will be made for each of the other profile changes.

Consistency will be assessed by repeating the same scenario and comparing the resulting portfolio characteristics, ranking and explanation. The evaluation will also record whether the agent uses the

required tools, completes the process within the configured iteration limit and returns a valid ranked set of recommendations. Explanations will be checked against the structured tool outputs to ensure that the stated reasons are supported by the profile, market signals and portfolio metrics used during the decision. These measures follow the agentic evaluation factors identified in Section 3.4.4, where task completion, rule following, consistency and correct tool interaction were identified as more useful than judging generated text alone.

The initial evaluation will use the single-agent design defined in Section 5.4. The two-agent alternative will only be tested if the scenario evaluation identifies problems with suitability, if repeated recommendations are inconsistent, or if explanations are not supported by the structured tool outputs. The same scenarios can then be repeated with the two-agent design to determine whether the additional reasoning step improves these areas.

6.5 System Performance Evaluation

Systems-performance testing will evaluate the completed application under load in the final RKE2 Kubernetes environment. The focus will be on how useful work changes as demand increases and how this relates to resource consumption and saturation within the application. The testing approach follows the benchmarking principles discussed by Gregg [X], including workload characterisation, progressively increasing load, repeated measurements and observing the system while each workload is running. The user-request and market-data paths will first be evaluated independently before being combined, allowing the limits of each workload to be established and then showing whether they interfere when operating at the same time.

6.5.1 User-Request Performance

The complete recommendation path will be tested using progressively increasing request concurrency, following the ramping-load approach described by Gregg [X]. Testing will begin at one concurrent request and increase through 2, 4, 8, 16 and 32, continuing to double, if necessary, until throughput stops increasing proportionally, or if p95 latency rises sharply or errors begin to occur. This allows the sustainable operating range and saturation point of the system to be identified without assuming its final capacity in advance.

For each workload, request throughput, error rate and p50, p95 and p99 end-to-end latency will be recorded. OpenTelemetry traces will additionally separate the request into its main processing stages, including user-context retrieval, agent and model activity, portfolio generation, analytics, suitability assessment and validation. CPU and memory usage will be monitored alongside relevant network and storage activity.

This stage-level information is particularly important because Experiment 1 identified a significant performance difference without enough tracing to determine exactly where the additional latency originated. The completed-system evaluation should therefore be able to connect any increase in latency to the processing stage or resource behaviour occurring at the same time. This also allows the final evaluation to go beyond benchmarking the application and identify which parts of the completed system limit its capacity. Where a performance problem is identified but its cause is not clear from the collected metrics and traces, further diagnosis can use the methodologies described by Gregg, such as the USE Method for identifying resource bottlenecks or Latency Analysis for locating the source of additional response time [X].

6.5.2 Market-Data Performance and Scalability

The event-driven market-data pipeline will be evaluated by increasing the amount of incoming market work and then varying the number of processing workers. The workload will initially use asset sets of 100, 500 and 1,000 equities, with the rate of incoming updates increased for each set until processing can no longer keep pace or sustained queue growth appears. This will show the operating range in which asset signals can remain current rather than only measuring the time required for one fixed workload.

The primary measures will be event throughput, update latency, queue depth, signal staleness, model-inference time, error rate and CPU and memory usage. Horizontal scalability will then be evaluated using initial worker counts of 1, 2, 4 and 8, with speedup and scaling efficiency calculated relative to the single-worker baseline. The purpose is not to expect perfectly linear scaling, but to determine how effectively additional compute increases processing capacity and where diminishing returns begin. This directly continues the scalability approach established during the architecture experiments, where additional consumers increased processing capacity but also increased the overall compute footprint.

6.5.3 Mixed Workload and Placement

The two workload paths will then be run simultaneously using the shared worker placement selected in Section 5.7. User-request performance will be compared with the independent user-request baseline, while market throughput, queue behaviour and signal staleness will be compared with the independent market-data baseline. Resource utilisation and saturation will be monitored to determine whether either workload creates measurable contention for the other.

This test is particularly important because Experiment 3 found that isolated compute did not improve user-facing performance in the experimental environment, while also showing that the unexpectedly stronger shared-placement result could not be generalised. Shared placement will therefore remain the initial treatment for the completed system. If running both workloads together consistently increases user-request latency, reduces market-processing capacity or increases signal staleness compared with their independent baselines, the test will be repeated with isolated placement. The results can then show whether separating the workloads provides enough improvement to justify the additional resources and deployment complexity.

6.5.4 Reliability and Recovery

Reliability testing will focus on two potential failures that could realistically occur during operation. The first test will terminate a user-request pod while requests are still being processed to see how many fail and how quickly service becomes available again. The second test will terminate a market-processing worker with events still waiting in the queue. This will show whether those events are picked up again after the worker is replaced and whether any duplicate or inconsistent signal updates are created.

The PostgreSQL instance will also be restarted to confirm that persisted application data is retained after pod replacement and that the failure is captured by the monitoring and logging setup. These tests are not intended to provide a full chaos-engineering evaluation. They are included to confirm that the system meets NFR-04 by recovering from expected failures without losing important state or hiding the failure.

6.6 Evaluation Criteria and Requirements Traceability

The final assessment will combine the results from the previous sections and trace them back to the requirements defined in Section 2. Functional requirements will be assessed through the component,

integration and end-to-end tests, while the non-functional requirements will use the corresponding systems-performance measurements.

Requirement	Evaluation Method	Success Measure
FR-01–FR-09	Functional and End-to-End testing	Required workflows and tests pass
NFR-01 Latency	User-request testing	User-request latency and sustainable operating range established
NFR-02 Throughput	User and market workloads	Maximum sustainable processing capacity established for both workloads
NFR-03 Scalability	Worker scaling	Processing capacity increases as worker resources are added
NFR-04 Reliability	Controlled failures	System recovers from tested failures without loss of required persisted state or inconsistent market updates
NFR-05 Resource Usage	Resource monitoring	Resource behaviour and saturation points identified across the tested workload range
NFR-06 Data Processing	Market-data testing	Market processing keeps pace with incoming updates within its sustainable operating range
NFR-07 Observability	Performance and failure tests	Limiting stages and tested failures can be identified from collected telemetry

Table 6.3: Requirements Testing Overview

Not every non-functional requirement requires a fixed numerical pass threshold. The evaluation will instead establish the operating range; saturation point and scaling behaviour of the completed system where these depend on the final application and workload. Fixed acceptance criteria are used where the expected behaviour is already known, such as constraint compliance, functional correctness and preservation of required state. Any numerical limits required for later operational monitoring will be configured from the completed-system measurements rather than selected arbitrarily before implementation.

The system will ultimately be considered successful if it can produce valid, personalised and explainable recommendations while operating within these defined technical limits. Where a requirement is not met, the collected traces and resource measurements will be used to identify the limiting component and determine whether the issue relates to application logic, model cost, resource saturation or workload placement. This provides a direct link between the original system requirements, the final implementation and the conclusions drawn from the thesis evaluation.

7. Conclusion

The purpose of this project was to establish the foundation required to build the proposed portfolio decision-support system. This involved defining the system requirements, researching the main financial, agentic AI and cloud-performance considerations, and experimentally evaluating architectural decisions that could not be resolved confidently through literature alone. These findings were then combined to produce a complete system design and evaluation plan that aligns with the specifications defined earlier in the report.

The experimental results showed that no single architectural approach was best suited to every part of the system. For the synchronous user-request path, separating the application into microservices introduced clear latency, throughput and resource overhead, supporting a more compact initial design. The market-data workload behaved differently, with event-driven processing providing higher throughput under lighter inference workloads and responding strongly to horizontal scaling as additional consumers were introduced. The later experiments also showed that workload placement and serverless deployment were more dependent on the final environment and usage pattern. Compute isolation did not improve performance under the tested conditions, while scale-to-zero reduced idle resource use at the cost of significant cold-start latency. Overall, these results primarily supported the main idea of choosing the architecture around the requirements of each workload rather than applying the same approach across the entire system.

These findings were combined with the literature review to produce a hybrid system design that reflects the different requirements of the two main workloads. The user-request path will use a single decision application, with its supporting tools kept as logically separated internal modules to avoid unnecessary communication overhead. While market-data processing will use an event-driven pipeline with independently scalable workers. A single tool-using agent will coordinate the recommendation process, with portfolio generation, calculations and constraint validation handled through deterministic supporting tools rather than being left entirely to the agent. The completed application will then be containerised and deployed on the provided RKE2 Kubernetes environment with the monitoring required to observe its behaviour during operation. The design therefore provides a clear starting architecture while still allowing additional scaling, isolation or deployment complexity to be introduced if later testing shows that it provides a meaningful benefit.

The final part of the project defined how the completed system will be evaluated during the thesis. This includes testing the core functionality, predictive models, recommendation suitability and constraint compliance, as well as the wider systems performance of the deployed application. The evaluation will therefore consider both whether the system produces suitable personalised recommendations and whether the underlying cloud-native design can support them efficiently in terms of latency, throughput, scalability, resource use and reliability. With the specification, design and evaluation plan now established, the main decisions required before development have been made. The next stage is therefore to implement the proposed system and evaluate it using the approach defined in this report.

8. References

[1] F. J. Fabozzi and H. M. Markowitz, “The Theory and Practice of Investment Management: Asset Allocation, Valuation, Portfolio Construction, and Strategies, Second Edition,” Wiley, 2011.

[2] S. D. Stewart, C. D. Piros and J. C. Heisler, “Portfolio Management,” Wiley, 2019.

[3] P. Sironi, “Fintech Innovation,” Wiley, 2016.

[4] Financial Industry Regulatory Authority, “2111. Suitability,” FINRA, [Online]. Available: https://www.finra.org/rules-guidance/rulebooks/finra-rules/2111.

[5] European Securities and Market Authority, “MiFID II: Article 25 Assessment of suitability and appropriateness and reporting to clients,” ESMA, [Online]. Available: https://www.esma.europa.eu/publications-and-data/interactive-single-rulebook/mifid-ii/article-25-assessment-suitability-and.

[6] C. Institute, M. G. McMillan, J. E. Pinto, W. L. Pirie and G. V. d. Venter, “Investments: Principles of Portfolio and Equity Analysis,” Wiley, 2011.

[7] M. Treveil, N. Omont, C. Stenac, K. Lefevre, D. Phan, J. Zentici, A. Lavoillotte, M. Miyazaki and L. Heidmann, “Introducing MLOps,” O'Reilly Media, Inc, 2020.

[8] C. Huyen, “Designing Machine Learning Systems,” O'Reilly Media, Inc, 2022.

[9] S. Jansen, “Machine Learning for Algorithmic Trading - Second Edition,” Packt Publishing, 2020.

[10] H. Tatsat, S. Puri and B. Lookabaugh, “Machine Learning and Data Science Blueprints for Finance,” O'Reilly Media, Inc, 2020.

[11] M. Peixeiro, “Time Series Forecasting in Python,” Manning Publications, 2022.

[12] MLflow, “MLflow: AI Engineering Platform for LLMs, Agents, & Models,” MLflow, [Online]. Available: https://mlflow.org/docs/latest/ml/.

[13] R. Crowe, H. Hapke, E. Caveness and D. Zhu, “Machine Learning Production Systems,” O'Reilly Media, Inc, 2024.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022.

[15] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger and C. Wang, “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” 2023.

[16] H. Yang, B. Zhang, N. Wang, C. Guo, X. Zhang, L. Lin, J. Wang, T. Zhou, M. Guan, R. Zhang and C. D. Wang, “FinRobot: An Open-Source AI Agent Platform for Financial Applications using Large Language Models,” 2024.

[17] Y. Xiao, E. Sun, D. Luo and W. Wang, “TradingAgents: Multi-Agents LLM Financial Trading Framework,” 2024.

[18] Z. Xiang, L. Zheng, Y. Li, J. Hong, Q. Li, H. Xie, J. Zhang, Z. Xiong, C. Xie, C. Yang, D. Song and B. Li, “GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning,” 2024.

[19] S. Yao, N. Shinn, P. Razavi and K. Narasimhan, “τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains,” 2024.

[20] Cloud Native Computing Foundation, “Cloud Native Definition,” CNCF, [Online]. Available: https://www.cncf.io/about/who-we-are/.

[21] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin and L. Safina, “Microservices: yesterday, today, and tomorrow,” 2016.

[22] D. Faustino, N. Gonçalves, M. Portela and A. R. Silva, “Stepwise Migration of a Monolith to a Microservices Architecture: Performance and Migration Effort Evaluation,” 2022.

[23] Y. Gan, Y. Zhang, D. Cheng, A. Shetty, P. Rathi, N. Katarki, A. Bruno, J. Hu, B. Ritchken, B. Jackson, K. Hu, M. Pancholi, Y. He, B. Clancy, C. Colen, F. Wen, C. Leung, S. Wang and L. Zaruvi, “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud and Edge Systems,” 2019.

[24] N. C. Mendonça, C. Box, C. Manolache and L. Ryan, “The Monolith Strikes Back: Why Istio Migrated From Microservices to a Monolithic Architecture,” IEEE, 2021.

[25] Kafka, “Introduction,” Kafka, [Online]. Available: https://kafka.apache.org/43/getting-started/introduction/.

[26] CNCF, “CloudEvents: A specification for describing event data in a common way,” CNCF, [Online]. Available: https://cloudevents.io/.

[27] J. Arafat, F. Tasmin and S. Poudel, “Next-Generation Event-Driven Architectures: Performance, Scalability, and Intelligent Orchestration Across Messaging Frameworks,” 2025.

[28] R. Laigner, A. C. Almeida, W. K. G. Assunção and Y. Zhou, “An Empirical Study on Challenges of Event Management in Microservice Architectures,” 2024.

[29] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica and D. A. Patterson, “Cloud Programming Simplified: A Berkeley View on Serverless Computing,” 2019.

[30] K. Djemame and B. Norton, “Serverless Computing: An Empirical Evaluation of Application Performance and Power Consumption in Knative,” ACM, 2025.

[31] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill and S. Uhlig, “Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions,” ACM, 2023.

[32] T. Fifield, D. Fleming, A. Gentle, L. Hochstein, J. Proulx, E. Toews and J. Topjian, “OpenStack Operations Guide,” O'Reilly Media, Inc, 2014.

[33] B. Burns, J. Beda, K. Hightower and L. Evenson, “Kubernetes: Up and Running, 3rd Edition,” O'Reilly Media, Inc, 2022.

[34] B. Burns, E. Villalba, D. Strebel and L. Evenson, “Kubernetes Best Practices, 2nd Edition,” O'Reilly Media, Inc, 2023.

[35] B. Gregg, “Systems Performance Enterprise and the Cloud - Second Edition,” Pearson, 2020.

[36] B. Beyer, C. Jones, C. Leng, D. Huska, J. Petoff and N. R. Murphy, “Site Reliability Engineering - Second Edition,” O'Reilly Media, Inc., 2026.